



OPEN
Compute Project



OPEN CLUSTER DESIGNS ALIGNED AI TRAINING FABRIC REFERENCE ARCHITECTURE

Author (s) :

Art Fewell, Hedgehog Inc.

Arun Solleti, Celestica Inc.

Date: April 2026

Version History

Date	Version #	Author	Description
27 MAR 2026	1.0	Art Fewell & Arun Solleti	Initial Draft
10 APR 2026	1.1	Art Fewell & Arun Solleti	Updated with review comments
26 APR 2026	1.2	Art Fewell & Arun Solleti	Updated with Steering Committee Feedback

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

Executive Summary

Open cluster designs for AI are of growing importance to the industry as organizations scale distributed training infrastructure from hundreds to thousands of accelerators. The Open Compute Project (OCP) Open Cluster Designs System Architectures [1][2] define modular building blocks for composing these clusters, but leave the network fabric — switch management, multi-tenancy, overlay policy, quality of service, and lifecycle automation — to implementers. Without a prescriptive network-centric reference, operators face fragmented guidance across vendors, ad-hoc tooling choices, and inconsistent operational practices that slow deployment and increase risk.

This document addresses that gap by describing the OCP Open Cluster Designs Aligned AI Training Fabric Reference Architecture (RA): an open, SONiC-based fabric managed by a Kubernetes-native control plane (Hedgehog) that provides declarative topology and overlay management, RoCE-oriented QoS, and automated lifecycle workflows. The architecture follows a modular scaling model built from OPG-M building blocks (Open Pod Group — modular pod groups of M xPUs) that compose into XOC-N clusters (xPU Open Cluster — scale-out clusters of N total xPUs), ranging from 64 to over 1,024 xPUs, in alignment with the OCP OPG-M and XOC-N System Architecture papers (January 2026) [1][2]. Lifecycle automation spans Day-0 Zero-Touch Provisioning, Day-1 connectivity-maps-as-code validation, and Day-2 continuous observability, ensuring consistent and repeatable operations at scale on OCP-compliant hardware such as the Celestica DS5000.

The RA provides composition variants across rail-optimized and single-homed Clos wiring patterns, single-plane (1×400G) and dual-plane (2×400G) configurations, and both air-cooled and liquid-cooled deployments at any rack density — the same topology, overlays, and control plane apply throughout. Each composition ships with connectivity maps, bills of materials, and Hedgehog Kubernetes CRDs as ready-to-deploy artifacts. A companion XOC-Virtual composition lets operators experience the full deployment and operational lifecycle in

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

Hedgehog's virtual lab (VLAB) — running the same controller and Broadcom Enterprise SONiC used in production — before committing to physical infrastructure.

The architecture has been validated in production: FarmGPU, a neocloud operator and OCP community leader, deployed and currently operates an XOC-512 cluster of NVIDIA B200 nodes using this RA's Hedgehog/Celestica topology. Independent performance analysis by SemiAnalysis confirmed that the fabric outperformed several alternative network solutions [12][13][14]. Multiple additional operators have deployed compositions from this RA at various size tiers, demonstrating its applicability across a range of deployment contexts.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

Table of Contents

Version History	2
1. Introduction	9
2. Compliance with Open Compute Project Tenets	11
2.1. Openness	11
2.2. Efficiency	12
2.3. Impact	12
2.4. Scale	13
2.5. Sustainability	13
3. Scope	14
3.1. Cooling and Density Independence	14
3.2. Target Compute Form Factor	14
4. Architecture Overview	17
4.1. Key Concepts: OPG and XOC	17
4.2. Network-Centric Perspective and Baseline Assumptions	19
4.3. OPG-M Network Building Blocks	20
4.4. XOC-N Tier Compositions	21
4.5. Topology Options (Standard, Rail-Optimized, Planes)	22
4.6. Scheduling and Tenancy Guidance	23
4.6.1. First Hop Domains (Single-homed Clos)	23
4.6.2. First-Hop Rail Domains (Rail-Optimized)	24
4.6.3. Placement Recommendations	25
4.7. Control Plane (Hedgehog) and Data Plane (SONiC)	26
4.7.1. Control Plane Components	27
4.7.2. Data Plane Components	27

Date: April 2026


 This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

4.8.	Logical Abstractions	28
4.9.	Network Profiles	28
4.10.	Operations Alignment	28
5.	Rail-Optimized Fabric Topology	29
5.1.	Definition and Purpose	29
5.2.	Critical Classification: Switch Counts Unchanged	29
5.3.	Rail Concept (NIC-GPU Affinity)	30
5.4.	Wiring Allocation at Different Scale	31
5.5.	Dual-Plane: Doubling the First-Hop Rail Domain	31
5.6.	Deterministic Mapping Scheme	33
5.7.	Single-homed Clos vs Rail-Optimized (Summary)	35
5.8.	Cabling Plant Considerations	36
6.	Single-Plane and Dual-Plane Fabric Options	38
6.1.	Overview	38
6.2.	Dependency: Server NIC Port Count	38
6.3.	Single-Plane Architecture	39
6.4.	Dual-Plane Architecture	39
6.5.	Dual-Plane Considerations	39
6.6.	Trade-offs and Selection Criteria	41
6.7.	Rack-Level Wiring by Density and Cooling	43
6.8.	Wiring and Operational Considerations	43
7.	Overlay and Policy	43
7.1.	Overlay Model (VPC → VRF/VNI)	43
7.2.	Virtual Private Cloud (VPC) API Resources and Policy Control	44
7.3.	Per-Network Considerations	45
7.4.	RoCE and QoS Approach	45
7.5.	Host and NIC Performance Tuning	46

7.6.	Border Connectivity	46
8.	Lifecycle and Automation	47
8.1.	Day 0: Initial Deployment	47
8.1.	Day 1: Configuration	48
8.2.	Day 2: Operations	50
9.	Validation Plan	52
9.1.	Objective and Scope	52
9.2.	Methods	53
9.3.	Success Criteria	54
9.4.	Limitation and External References	55
9.5.	Packaging and Reproducibility	55
10.	Conclusion	55
11.	Glossary	57
12.	References	67
13.	License	68
13.1	Creative Commons - This is the suggested language.	68
13.1.	Copyright License - This is the suggested language.	69
14.	About Open Compute Foundation	69
Appendix A. Composition Variant Summary		70
Repository Structure		70
OPG Building Blocks		72
XOC Cluster Compositions		73
Topology Types		74
Assets Per Variant		74
Variant Naming Convention		76
XOC-Virtual (VLAB Composition)		76

Date: April 2026


 This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

How to Use

77

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

1. Introduction

This document describes the Open Cluster Designs Aligned AI Training Fabric Reference Architecture (RA), a deployment-ready, network-centric contribution by Hedgehog and Celestica. The RA provides detailed specifications for topology patterns (single-homed Clos and rail-optimized wiring), the control plane, RDMA over Converged Ethernet (RoCE)-oriented Quality of Service (QoS) guidance (lossless versus best-effort), and lifecycle workflows (Day-0 Zero-Touch Provisioning (ZTP), Day-1, and Day-2). Non-network components (compute, storage, management) are treated per the examples presented in the OCP OPG-M and XOC-N system architecture papers when needed to explain network provisioning, unless otherwise noted[1][2]. The structure and tone follow OCP guidelines to ensure technical precision and reviewer familiarity.

Two OCP companion specifications provide essential context. The OPG-M System Architecture [1] defines the Open Pod Group for M xPUs (OPG-M), a modular building block: OPG-128, for example, consists of 16 eight-xPU servers together with their networking, storage, and management resources. The XOC-N System Architecture [2] defines the xPU Open Cluster for N xPUs (XOC-N), which composes one or more OPG pods under a shared spine and aggregation layer into a scale-out cluster of N xPUs. This Reference Architecture provides the network-centric complement to those specifications, both for the exact OPG-128 example topologies presented as well as a number of additional compositions. The network design is independent of cooling modality (air or liquid) and rack density - although out-of-band network connectivity maps provided have been extended to provide connectivity for PDU's for air-cooled densities, and PDU's, CDU's and Heat Exchangers for liquid cooled densities.

This RA is grounded in production experience. FarmGPU, a Neocloud operator that co-leads the OCP "Scaling AI Clusters at Neoclouds" workstream, deployed and operates an XOC-512 B200 cluster using the Hedgehog/Celestica fabric described here. SemiAnalysis independently analyzed FarmGPU's xPU infrastructure and reported that the network fabric outperformed several other network solutions in their testing [12][13]. FarmGPU's rapid deployment —

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

documented in their public blog series [14] - demonstrates the operational efficiency of the declarative, ZTP-driven lifecycle model described in this document. FarmGPU is highlighted as a featured partner and public reference for this Reference Architecture; additional operators deploy compositions from this RA across multiple size tiers.

Hedgehog AI Network, an open-source (Apache 2.0) Kubernetes-native fabric management platform, provides the control plane for this RA. Operators declare intent through a declarative model of Kubernetes Custom Resource Definitions (CRDs) that express both physical topology (the "wiring diagram") and logical overlays(VPCs). Hedgehog manages SONiC-based switches via Kubernetes CRDs and continuous reconciliation, treating switches as data-plane elements, which reduces operational risk and enables GitOps workflows. This approach supports a multi-vendor posture via switch profiles, consistent with OCP principles. Hedgehog currently operates with Broadcom's SONiC distribution on supported switch platforms.

Celestica's DS5000 (800G) serves as the reference switch platform, representing the OCP-compliant 51.2Tbps switch described in the OPG-M specification; the design remains vendor-heterogeneous and portable via switch profiles. Scale-out NICs (examples provided with NVIDIA ConnectX-6, ConnectX-7 and ConnectX-8 with specific tuning and optimization guidance) connect each xPU to the fabric. For external connectivity, border functions may use the Celestica DS3000 in conjunction with a distributed, open-source Hedgehog gateway on x86, providing policy-driven NAT/PAT and firewall services. Out-of-band networks are not managed by Hedgehog to avoid circular dependencies. Interfaces are standard: Kubernetes APIs, gRPC Network Management Interface (gNMI) on devices, and OpenBMC/Redfish for platform context.

This RA is accompanied by a companion repository [15] that provides deployable composition assets for every size tier and variant described in this document. Each variant folder contains a connectivity map (connectivity-map.csv) providing end-to-end device and port mapping, a topology authoring plan (topology-map.yaml), Hedgehog Kubernetes CRD wiring definitions that serve as the complete initial fabric configuration, NetBox-importable inventory, and draw.io

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

topology diagrams. The Hedgehog wiring CRDs make literal the concept of connectivity-maps-as-code: operators can cable switches as documented, feed the provided wiring YAML into the Hedgehog fabricator tool (hhfab), and the controller will configure the entire fabric through zero-touch lifecycle management — no manual switch CLI required. The VPC API CRDs then provide a simplified interface for allocating network connectivity across the running fabric. The companion repository [15] also includes host and NIC performance tuning recommendations contributed by FarmGPU based on their production XOC-512 deployment. The repository is organized by size tier (OPG-64 through OPG-512 and XOC-64 through XOC-1024), with tokenized variant names for easy navigation and grep-ability.

The remainder of this document covers scope and baseline assumptions, OPG-M and XOC-N compositions from a network perspective, topology patterns (single-homed Clos and rail-optimized) with single- and dual-plane variants, connectivity-maps-as-code, lifecycle and observability, and validation methods. Hardware baselines, wiring diagrams, port maps, and example Kubernetes CRDs are provided as assets or in follow-on sections.

2. Compliance with Open Compute Project Tenets

This Reference Architecture builds on existing and extensive OCP work and aligns naturally with the OCP Tenets. In summary:

2.1. Openness

The contribution is open by design and implementation. Hedgehog AI Network, the control plane for this Reference Architecture, is fully open source (Apache 2.0). The Hedgehog control plane operates data plane elements that run on SONiC, the Linux Foundation's open-source network operating system, and exposes Kubernetes CRDs to define physical topology and logical overlays. This allows adopters to inspect, extend, and automate the system using standard tooling and workflows such as GitOps without proprietary interfaces.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

Hardware lock-in is avoided through a multi-vendor approach aligned with OCP principles. Switch profiles abstract platform-specific capabilities so that third parties can substitute OCP-compliant switches while preserving the same logical design and artifacts. The document explicitly aligns with OCP projects and specifications such as Open Rack version 3 (ORv3) and encourages collaboration and extension through declarative artifacts (CRDs, connectivity-maps-as-code) that can be modified and versioned by the community.

2.2. Efficiency

Operational efficiency is achieved by using a Kubernetes-native control plane with declarative Kubernetes CRDs rather than device-specific CLI or templated push models. The Hedgehog controller continuously reconciles desired state to actual state on SONiC-based switches, reducing configuration drift and the need for manual intervention. This improves safety and shortens iteration cycles for common operations (for example, tenant onboarding via VPC abstractions, port/NIC mapping at bring-up) using standardized APIs and automation.

Deployment efficiency is realized through size-tiered templates and profiles aligned to OPG-M/XOC-N compositions (for example, OPG-64/128/256/512 building blocks composing XOC-64 through XOC-1024 tiers). Pre-validated topology, overlay, and QoS defaults reduce turn-up time while preserving the ability to tune. Cost efficiency follows from using open-source control software and OCP-compliant hardware with competitive supply chains, while the ability to reuse the same logical design across vendors via switch profiles preserves engineering effort across lifecycle transitions.

2.3. Impact

This Reference Architecture makes XOC-N practices directly adoptable by organizations beyond hyperscalers. By providing complete, practical artifacts - declarative topology and overlay definitions, connectivity-maps-as-code, and lifecycle workflows - enterprises with lean IT staff can plan, procure and deploy AI fabrics with clear, reproducible steps. The alignment to

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

OCP specifications and reliance on open software broaden accessibility and help standardize operational practices in the ecosystem.

The approach also accelerates time-to-deployment: operators can start with validated size tiers and network profiles and evolve configurations as needs change, without re-authoring device-level configurations. Using OCP hardware and open-source software supports global supply options and encourages convergence on interoperable patterns across contributors and adopters.

2.4. Scale

Scaling follows the XOC-N model of composing OPG-M building blocks into clusters: OPG-64/128/256/512 building blocks compose into XOC-64/128/256/512/1,024 tiers. Each tier provides a non-blocking Clos fabric, and templates for each tier provide additive capacity without redesign. Network profiles (Minimal four-network and Full five-network) allow right-sizing to workload requirements while maintaining consistent control-plane semantics.

Day-0 ZTP and Day-1/Day-2 automation reduce operator effort as environments grow. Because the state is declarative and continuously reconciled, expanding or modifying the fabric (for example, adding leaves/spines or adjusting overlay policies) requires minimal manual touch and preserves consistency across failure and upgrade events. Management interfaces are standard (Kubernetes APIs; gNMI on devices), enabling integration with existing tooling at scale.

2.5. Sustainability

The design emphasizes efficient use of resources and long-lived operability. Building on OCP hardware provides a baseline for efficiency and serviceability. Right-sizing via network profiles and size-tiered topologies helps avoid over-provisioning. Automation (ZTP; continuous reconciliation) reduces unnecessary site visits and configuration churn, and observability pipelines integrate with widely available Loki, Grafana, Tempo, Mimir (LGTM) components to focus remediation on real issues rather than broad guesswork.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

Open, modular software and multi-vendor hardware flexibility support reusability and longevity: logical designs and operational artifacts can persist across hardware generations and vendors, reducing rewrite waste and extending useful service life. Subsequent sections document quantitative considerations and configuration practices so operators can make informed trade-offs consistent with OCP's sustainability goals.

3. Scope

This Reference Architecture provides a prescriptive model for scale-out Ethernet fabrics aligned with OCP Open Cluster Designs. It is network-centric: topology, control plane, quality of service (QoS), telemetry, and lifecycle workflows are specified in detail. The design applies equally to air-cooled and liquid-cooled deployments; cooling modality and rack density do not affect the fabric architecture. Non-network components (compute, storage, management) are treated per the examples presented in the OCP OPG-M and XOC-N system architecture papers when needed to explain network provisioning, unless otherwise noted[1][2].

3.1. Cooling and Density Independence

This RA's network architecture is identical for air-cooled (typically ~2 xPU-node servers per rack) and liquid-cooled (typically ~8 xPU-node servers per rack) deployments. The same topology, control plane, overlays, and QoS apply regardless of cooling modality.

3.2. Target Compute Form Factor

This RA targets xPU-node servers -- servers containing eight xPUs (such as GPUs) where the high-bandwidth scale-up interconnect (such as NVLink or NVSwitch) is entirely within the server chassis. Each xPU-node has eight GPU-affinitized scale-out NICs connecting to the Ethernet fabric; composition variants provide single-plane configurations with 1×400GbE per NIC and dual-plane configurations with 2×200GbE or 2×400GbE per NIC. This RA treats each xPU-node as the atomic compute unit for scale-out networking.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

Transceiver selection in the provided assets deliberately targets flexible-reach options (e.g., DR/SR classes) that should span typical data-center layouts at the stated sizes; operators should validate optic types against measured cable lengths and, where needed, substitute DACs/AECs or alternate optics to optimize for power, cost, or reliability once rack locations and cable routes are finalized.

Baseline assumptions are defined in Architecture Overview (see Table '[Baseline Assumptions](#)').

In Scope:

- **Network topology:** Leaf-spine (folded-Clos) with single-homed server-to-leaf cabling, folded Clos with rail-optimized server-to-leaf cabling (a wiring constraint that preserves rail locality; same switch counts as single-homed Clos), single- and dual-plane options; DS5000 as reference platform (vendor-agnostic via switch profiles).
- **Scaling tiers:** OPG-M building blocks (OPG-64/128/256/512) and XOC-N compositions (XOC-64 through XOC-1024+); guidance for smaller deployments and partial population.
- **Control plane:** Hedgehog (Kubernetes-native), Kubernetes CRD-based topology and overlay definitions, continuous reconciliation to SONiC-based switches.
- **QoS and traffic engineering:** One-parameter RoCE enablement, Differentiated Services Code Point (DSCP) class selection, Priority Flow Control (PFC) / Explicit Congestion Notification (ECN) configuration, traffic-class profiles.
- **Lifecycle workflows:** Zero-Touch Provisioning (ZTP) via Open Network Install Environment (ONIE), Day-1 intent, Day-2 operations and observability with Grafana Alloy collection for Loki Grafana Tempo Mimir (LGTM).
- **Connectivity budgets:** xPU-node NIC requirements, storage connectivity budgets, border/external connectivity patterns.

Out of Scope:

- **Compute specifics:** Exact server models/vendors, OS provisioning methods, and scale-up interconnects (such as NVLink and NVSwitch, which are internal to the xPU-node or rack). Scale-up is orthogonal to the scale-out Ethernet fabric this RA specifies.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](#).

- **Storage specifics:** Exact storage node counts, capacity sizing, and storage software stack.
- **Facility:** Power distribution, cooling system design, and physical rack layout beyond connectivity. Cooling modality is orthogonal to this RA -- the same network design applies to air-cooled and liquid-cooled deployments.
- **Out-of-band management infrastructure:** Hedgehog does not manage the out-of-band network (to avoid a circular dependency where the management tool depends on the network it manages). OoB switches (such as DS1000) are present but outside this RA's control plane.
- **Rack-spanning scale-up architectures:** Designs where the scale-up interconnect spans multiple racks imply different networking assumptions and are out of scope for the scale-up portion. The scale-out Ethernet fabric described here remains applicable for inter-node traffic in those deployments.

Network Profiles

This RA defines two network profiles. These map to the network types defined in the Open Cluster Designs for AI specifications (XOC-N [2]). For the detailed network profile mapping table, see the Architecture Overview section.

- **Minimal profile (4 networks):** SO-C + Storage (converged), back-end (scale-out), in-band management, out-of-band management (not Hedgehog-managed).
- **Full profile (5 networks):** SO-C, Storage, Scale-out CPU (SO-C), back-end, in-band management; out-of-band management present but not Hedgehog-managed.

Relationship with Other OCP Contributions

This RA provides the network-centric view of OPG-M (leaf-layer attachment, rail allocation, QoS) and XOC-N (spine layer, multi-OPG composition, scaling tiers). It assumes 8 xPUs per xPU-node, matching the OPG-M exemplar form factor. It uses SONiC as the NOS on switches

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

and assumes OpenBMC/Redfish for hardware management context. Rack-scale foundations such as Open Rack version 3 (ORv3) are assumed where applicable.

Composition variants in the companion repository [15] provide concrete wiring, bills of materials, and Hedgehog Kubernetes CRDs for OPG-64/128/256/512 and XOC-256/512/1024 in both single-homed Clos, rail-optimized, and dual-plane rail optimized patterns.

4. Architecture Overview

This section presents the network-centric architecture of an Open Cluster Designs aligned AI training fabric composed from OPG-M building blocks into XOC-N clusters, managed by Hedgehog. It establishes baseline assumptions, composition tables, topology options (single-homed Clos, rail-optimized, single-/dual-plane), control/data-plane roles, and scheduling guidance.

4.1. Key Concepts: OPG and XOC

Understanding two building blocks is essential:

- **OPG pod** (Open Pod Group for M xPUs, OPG-M): A modular cluster building block consisting of xPU-node servers plus their networking, storage, and management resources. The "M" is the total xPU count -- for example, OPG-128 contains 16 eight-xPU servers (128 xPUs total). An OPG defines the leaf-layer attachment and rail allocation.
- **XOC cluster** (xPU Open Cluster for N xPUs, XOC-N): A complete scale-out cluster composed of one or more OPG pods connected under a shared spine layer. The "N" is the total xPU count -- for example, XOC-512 composes four OPG-128 pods (512 xPUs total). The XOC adds spine/aggregation and north-south connectivity.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

Example: A site deploys an XOC-256 cluster by connecting two OPG-128 pods (16 xPU-node servers each, 128 xPUs each) under a shared spine layer, yielding 256 xPUs with consistent topology and control-plane semantics.

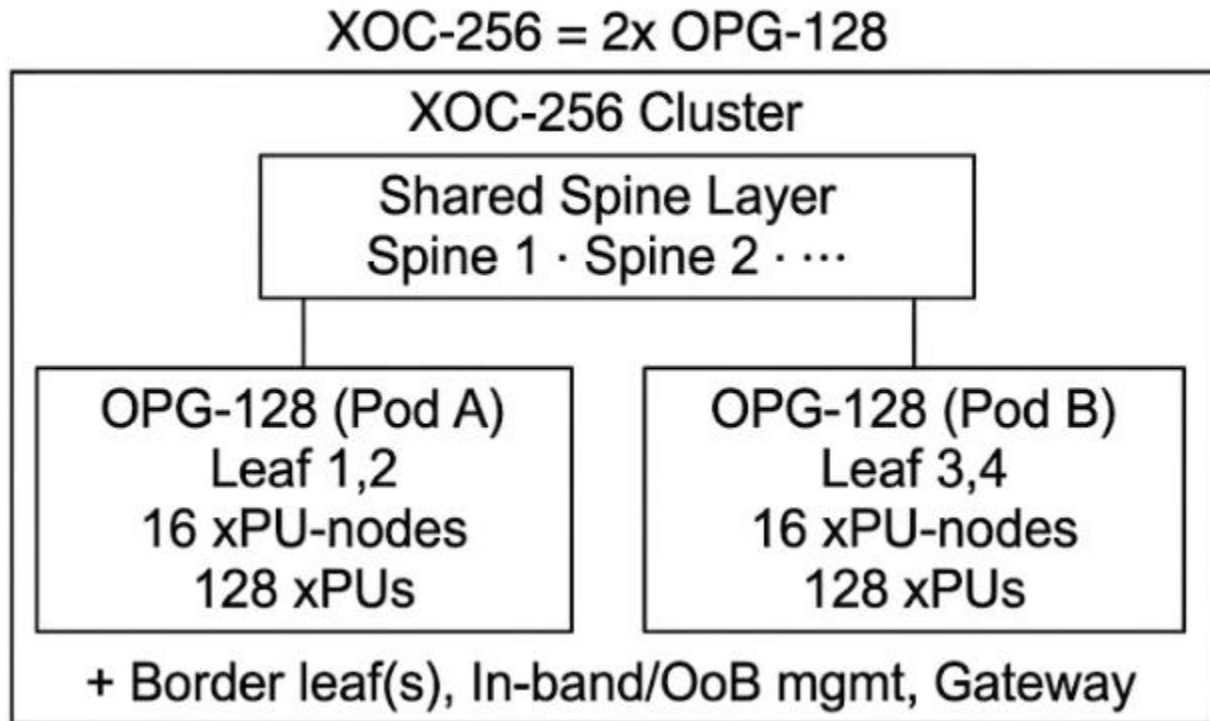


Figure 1: XOC-256 composed from two OPG-128 building blocks under a shared spine layer. Each OPG pod contains leaf switches and xPU-nodes; the XOC adds spine/aggregation and north-south connectivity.

These terms originate from the OCP Open Cluster Designs System Architectures: OPG-M System Architecture [1] and XOC-N System Architecture [2].

4.2. Network-Centric Perspective and Baseline Assumptions

This RA specifies the scale-out Ethernet fabric. Non-network components are assumed OCP-compliant and comparable to the examples presented in the OCP OPG-M and XOC-N system architecture papers, unless otherwise noted [1][2].

- **xPU-node:** The server form factor for this architecture. Scale-up resources are contained within the enclosure (eight xPUs per node) - while the canonical examples specify air cooled form factors, this reference architecture works equally well with air or liquid cooled xpu servers in the 8 xpu per server form factor.
- **Scale-out Fabric:** The Ethernet fabric responsible for interconnecting all xPU-nodes. The DS5000 switch platform represents the OCP-compliant 51.2 Tbit switch described in the OPG-M specification [1].
- **SO-C (Front-end) Network:** Connectivity for CPU/gateway/ingress services. Each xPU-node connects via BlueField-3 (or equivalent), providing 2×200G across two leaf switches. Multi-homing for front-end connectivity uses Hedgehog's HostBGP feature [HH-DOCS-REF]: a host-side FRR (Free Range Routing) container (``ghcr.io/githedgehog/host-bgp``) automatically establishes unnumbered eBGP sessions with redundant leaves, and the host advertises its Virtual IPs (VIPs) as /32 routes into the fabric. The fabric then uses ECMP to load-balance and fail over across the redundant leaf paths - eliminating manual coordination, peer-link overhead, and L2 protocol complexity. HostBGP is enabled declaratively by setting ``hostBGP: true`` on the relevant VPC subnet. Typically, best-effort with QoS protection for control traffic.
- **Storage Network:** Connectivity for storage servers/arrays; speed and link count vary by design; QoS applied to storage flows where required.
- **In-band Management Network:** Cluster management and telemetry; managed by Hedgehog; Celestica DS2000 used in examples.
- **Out-of-band (OoB) Management Network:** BMC/hardware management; not managed by Hedgehog to avoid circular dependencies. Celestica DS1000 used in examples. Port/connectivity mappings are provided in the published assets.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

- **Cooling modality** is orthogonal to this design -- the same topology, control plane, overlays, and QoS apply to both air-cooled and liquid-cooled deployments.

Baseline assumptions:

Parameter	Value	Notes
xPUs per server	8	xPU-node form factor
Scale-out NICs per server	8	NIC1 ... NIC8, GPU-affinitized
Scale-out NIC port Speed	2x200G (CX-7), 1x400G (CX-7), 2x400G (CX-8)	Connectivity Maps, Hedgehog configuration (wiring diagram) files and other assets are provided in support of multiple scale-out port speed options
DS5000 total ports	64@800G 128@400G 256@200G	Each port can be broken out into 2x400G, 4x200G and other patterns
DS5000 uplinks (to spines)	32@800G	50% of ports
DS5000 downlinks (to servers)	32@800G, 64@400G, 128@200G	50% of ports
Servers per leaf	Varied - see notes	Varies depending on breakout configuration and single vs dual plane, single-homed Clos vs rail-optimized
xPUs per leaf	64@400G 128@200G	Varies based on scale-out NIC port speed

4.3. OPG-M Network Building Blocks

OPG-M is a modular cluster building block composed of xPU-nodes plus networking, storage, and management resources. The "M" denotes the total xPU count in the pod. The table below illustrates the OPG building blocks for the single-plane, 1x400GbE scale-out NIC configuration (one leaf serves 64 xPUs when each xPU-node connects via 8x400G):

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

OPG Size	Leaves	Servers (8xpu/node)	xPUs	Notes
OPG-64	1	8	64	Base unit (1 leaf)
OPG-128	2	16	128	2 leaves
OPG-256	4	32	256	4 leaves
OPG-512	8	64	512	8 leaves

Leaf counts change for other scale-out NIC configurations (for example, 2×200GbE or 2×400GbE dual-plane designs use different breakout patterns and port allocations). Each OPG pod is a building block, not a standalone deployment -- it requires XOC spine/aggregation and external connectivity to form a complete cluster.

The companion repository [15] provides fully specified variants for each OPG size across single-plane and dual-plane configurations -- complete with connectivity maps, bills of materials, Hedgehog Kubernetes CRDs, and topology diagrams. Each variant folder is self-contained and ready for operator review, procurement planning, or direct deployment.

4.4. XOC-N Tier Compositions

XOC-N composes one or more OPG-M building blocks into a cluster-level system -- adding a shared spine layer, north-south connectivity, and unified control-plane management across all constituent OPG pods. The table below shows XOC tier compositions for the single-plane, 1×400GbE configuration. Spine counts provide a non-blocking Clos fabric at each tier; the design reserves 50% of DS5000 ports as uplinks to spines, yielding full bisection bandwidth within the scale-out plane. Exact spine counts may be adjusted for external connectivity requirements.

For smaller compositions that consist of 1-3 leaf switches, Hedgehog provides the option to support a mesh configuration in which the leaf switches directly interconnect, providing a fully-nonblocking fabric without requiring separate spine switches. This does not require operators to maintain architecture and configurations for alternate topologies, Hedgehog abstracts the configuration and provides a consistent interface for tenant configuration.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

XOC Tier	Composition	Leaves	Spines	Servers	xPUs
XOC-64	1x OPG-64	1	2 or Mesh	8	64
XOC-128	1x OPG-128	2	2 or Mesh	16	128
XOC-256	2x OPG-128	4	2	32	256
XOC-512	4x OPG-128	8	4	64	512
XOC-1024	8x OPG-128	16	8	128	1024

For dual-plane and alternative NIC speed configurations, leaf and spine counts differ. Rather than duplicate tables for every combination, the companion repository [15] provides the complete topology details for all provided composition variants -- organized by size tier, wiring pattern, plane configuration, and cooling density -- so operators can navigate directly to the composition that matches their deployment requirements.

4.5. Topology Options (Standard, Rail-Optimized, Planes)

- Single-homed Clos:** A folded leaf-spine topology, aligned with the "Clos topology" cabling pattern defined in the OPG-M System Architecture [1], in which all eight scale-out NICs from each xPU-node connect to the same leaf switch. With the DS5000 at 1×400GbE per NIC, this places 8 xPU-nodes (64 xPUs) on a common leaf, forming a first-hop domain in which all traffic between those nodes – both intra-rail and cross-rail – is forwarded within the leaf switch and never traverses the spine. Note: the OPG-M specification refers to this as "Clos topology"; this RA uses the more specific term "single-homed Clos" because there are multiple different server-to-leaf cabling patterns that can represent a valid Clos topology – the distinction is in the server-to-leaf cabling pattern, not the topology itself.
- Rail-optimized wiring:** A cabling strategy that ensures all NICs within the same GPU-affinity index (a "rail") connect to the same leaf switch or a contiguous set of leaf switches. This is a wiring constraint only -- it does not alter switch count, port budgets, or bandwidth ratios compared to a single-homed Clos fabric. Rail-optimized wiring

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

materially reduces spine utilization by keeping scale-out exchanges leaf-local when tenants are placed within a common first-hop rail domain.

- **Single-plane vs. dual-plane:** Single-plane utilizes a single, unified Clos fabric for all scale-out traffic. Each GPU-affinitized NIC has one port (such as ConnectX-7, 1x400G). Dual-plane provides two isolated fabrics (Plane A and Plane B) to improve redundancy and allow maintenance without downtime. Each GPU-affinitized NIC has two ports (such as ConnectX-8, 2x400G), with one connecting to each plane.

4.6. Scheduling and Tenancy Guidance

Topology-aware tenant placement is one of the most effective tools for managing spine utilization in large AI clusters. Both single-homed Clos and rail-optimized wiring create natural placement boundaries — called "first-hop domains" — that operators can exploit to keep traffic off the spine.

4.6.1. First Hop Domains (Single-homed Clos)

In the single-homed Clos (single-homed) cabling pattern, all eight scale-out NICs from each xPU-node connect to the same leaf switch. With the DS5000 at 1×400GbE per NIC, this places 8 xPU-nodes (64 xPUs) on a common leaf, forming an 8-node **first-hop domain**. Within this domain, ALL traffic between nodes — both intra-rail and cross-rail — is forwarded within the leaf switch and never traverses the spine. Any tenant workload allocated entirely within a first-hop domain contributes zero spine traffic regardless of its communication pattern.

This property makes single-homed Clos particularly attractive for many multi-tenant environments. Consider an xPU-as-a-service provider where a large percentage of cluster leases are for 8 nodes or fewer: if tenancy can be allocated within first-hop domain boundaries, those tenants produce no spine traffic. The provider can still lease larger clusters that span multiple domains, but by keeping a significant fraction of tenancy within 8-node first-hop domains, overall spine congestion is reduced – enabling the cluster to scale to larger composition sizes with predictable performance.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

4.6.2. First-Hop Rail Domains (Rail-Optimized)

In a rail-optimized design, the natural placement unit is the **first-hop rail domain** – the set of xPU-nodes whose rail-n NICs all terminate on a common leaf (or contiguous set of leaves). Intra-rail traffic within a first-hop rail domain stays leaf-local; cross-rail traffic between xPU-nodes in the same rail domain may traverse the spine, depending on rail domain sizing.

Rail domain sizing is a design variable. The primary benefit from rail-optimized design stems from all servers in a given first-hop rail domain having all of their rail n NICs connected to the same leaf switch as the rail n NICs of all other servers within the same domain. This is exemplified in an OPG-512 composed with 64 xPU servers that use a 1x400G port per scale-out NIC (rail).

When using the 51T Celestica DS5000 Form factor, 64x400G ports are available for server connections per leaf switch (assuming 50% of ports used as uplinks). This results in a dedicated leaf switch per rail, all 64 servers have their rail-1 NIC connected to leaf-switch 1, their rail-2 NIC connected to leaf-switch 2 and so on, 1 leaf switch for each of the 8 rails. All of the intra-rail traffic between these 64 nodes will not traverse the spine. That does not mean you need to have to dedicate a leaf switch per rail, the defining element is that all nodes which have all of their rail n NICs on common leaf switch. For example, an OPG-256 could be composed with 4 leaf switches, where each switch handled 2 rails, and an OPG-128 with 2 leaf switches that each handled 4 rails. Even the single-homed traffic pattern could also be described as a rail-optimized pattern with the key difference being that the fewer switches you distribute rails across, the smaller the size of the first hop rail domain. Conversely a dual plane design can double the size of a 1st hop rail domain. As an example, consider the following rail domain sizes for single-plane topologies:

- **1× 8-leaf rail domain (64 xPU-nodes):** Each rail gets a dedicated leaf. The entire OPG is one first-hop rail domain — the largest possible with this NIC and switch form factor in a single plane design. Intra-rail traffic for the entire OPG is leaf-local. However, all cross-

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

rail traffic traverses the spine. This configuration maximizes intra-rail locality across the entire OPG. The spine-traffic reduction benefits any number of tenants or jobs sharing the domain, to the extent their traffic is intra-rail.

- **2× 4-leaf rail domains (32 xPU-nodes each):** Each leaf serves 2 rails. Cross-rail traffic between rails on the same leaf stays local; cross-rail traffic between rails on different leaves traverses the spine. Balances domain size with cross-rail locality.
- **4× 2-leaf rail domains (16 xPU-nodes each):** Each leaf serves 4 rails. The majority of cross-rail traffic stays leaf-local (only traffic between rails assigned to different leaves crosses the spine). This configuration provides smaller, more numerous placement units and retains good cross-rail locality within each domain.

The compositions submitted with this RA include OPG variants with 2-leaf, 4-leaf, and 8-leaf first-hop rail domains for single-plane designs, and up to 16-leaf first-hop rail domains for dual-plane designs. Operators should select domain sizing based on their expected workload mix: larger domains offer both maximum intra-rail locality and easier bin-packing for varied job sizes; smaller domains provide better cross-rail locality within each domain. The deciding factor is the expected traffic profile — specifically, the ratio of intra-rail to cross-rail traffic — rather than job size or tenant count alone.

4.6.3. Placement Recommendations

- **Single-homed Clos:** Allocate tenants/workloads within 8-node first-hop domains wherever possible. Workloads confined to a single domain produce zero spine traffic. Reserve cross-domain allocations for jobs that genuinely require more than 64 xPUs.
- **Rail-optimized:** Allocate tenants/workloads within first-hop rail domains. Packing a tenant, allocating workloads, and using within one domain and using rail-aware, hierarchical schedulers keeps intra-rail traffic leaf-local; spreading nodes across domains forces those flows through the spine.
- **Scheduler integration:** Integrate domain awareness into the workload scheduler (for example, Kubernetes node labels or SLURM partition definitions). Label nodes by their

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

first-hop domain (single-homed Clos) or first-hop rail domain (rail-optimized) so that the scheduler can prefer domain-local placement.

- **Composition example:** In an XOC-256 cluster composed of two OPG-128 pods, a four-node training job placed inside a single first-hop rail domain contributes minimal spine traffic. Placing one node per domain across both OPG pods forces all scale-out traffic through the shared spine layer.

4.7. Control Plane (Hedgehog) and Data Plane (SONiC)

Hedgehog Fabric consists of several key components, distributed between the Control Node and the Network devices. The following diagram breaks down the components of a mesh topology:

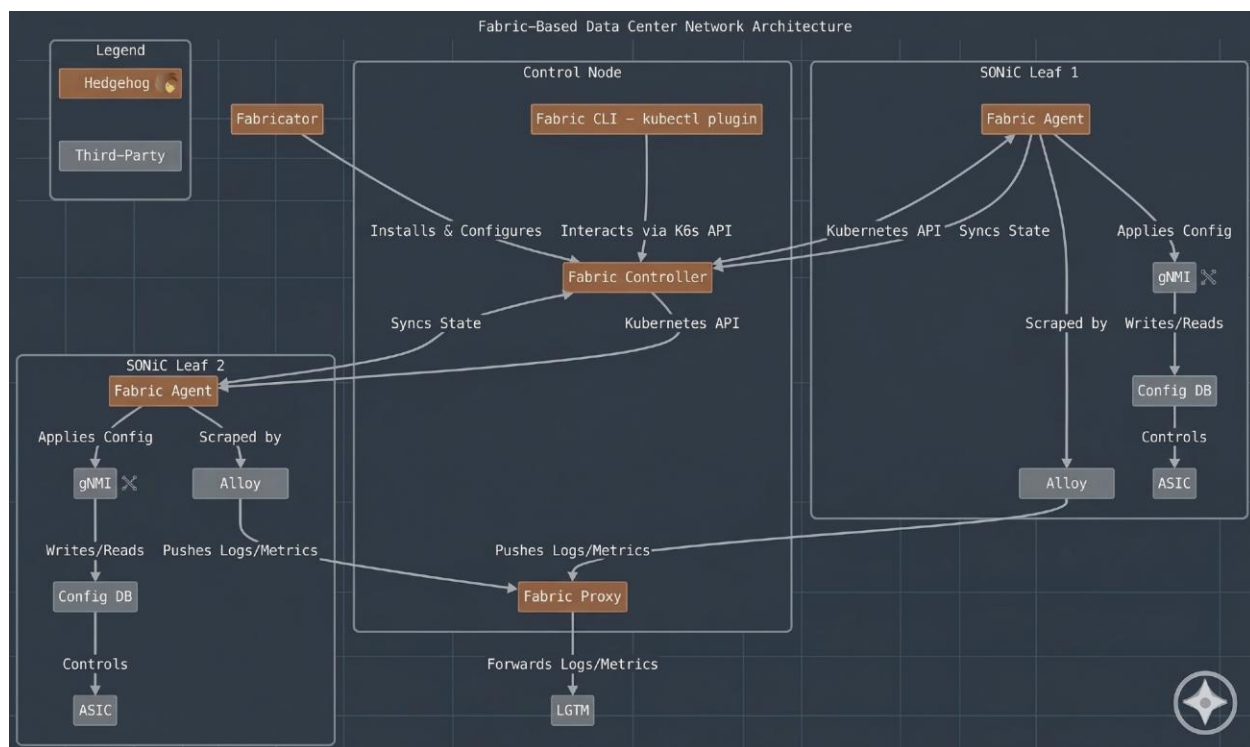


Figure 2: Hedgehog control-plane and data-plane architecture. Operators declare intent via Kubernetes CRDs; the Hedgehog controller computes desired state and reconciles it to SONiC-based switches through per-switch agents.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

4.7.1. Control Plane Components

Hedgehog controller (in Kubernetes) watches CRDs for topology and overlays, computes desired state, and reconciles to switches via agents. Declarative, GitOps-friendly model; continuous reconciliation corrects drift.

- Fabric Controller: The central control plane component that manages Fabric resources and configurations.
- Fabric CLI (kubectrl plugin): A kubectrl plugin that provides an easy way to manage Fabric resources.
- Fabric Proxy: A pod responsible for collecting logs and metrics from switches (via Alloy) and forwarding them to an external system.
- Fabricator: A tool for installing and configuring Fabric, including virtual lab environments.

4.7.2. Data Plane Components

SONiC runs on switches (like Celestica DS5000) and provides forwarding / routing. EVPN serves as the overlay control plane and VXLAN as the data plane. Interfaces include Kubernetes APIs for intent, gNMI for switch configuration, telemetry, and observability export to LGTM.

- Fabric Agent: Runs on each switch and applies configurations received from the control plane.
- Alloy: Collects logs and telemetry data from the switch.
- gNMI Interface: The main configuration API used by the Fabric Agent to interact with the switch.
- Config DB: The central database in SONiC responsible for maintaining switch configuration.
- ASIC: The hardware component handling packet forwarding.

The SONiC architecture presented here is a high-level abstraction, for simplicity.

4.8. Logical Abstractions

- **Wiring Diagram (CRDs):** *Switch*, *Server*, *Connection* define physical topology, port roles, and aggregation types (unbundled with HostBGP for L3 multi-homing, bundled/LAG, ESLAG, fabric links).
- **VPC Abstraction (CRDs):** *VPC*, *VPCAttachment*, *External*, *ExternalAttachment*, *ExternalPeering*, *VPCPeering* define overlays, attachments, and external connectivity.
- **Mapping:** VPC→VRF; Subnet→VNI; EVPN for control plane; VXLAN for data plane; symmetric IRB; Type-2 and Type-5 routes. On TH5 leaves, back-end networks may use L3VNI semantics to conserve L2 resources.

These Kubernetes CRDs provide a declarative, version-controlled interface for defining both physical wiring and logical network intent. Operators apply them with `kubectl apply` or `hhfctl`, and Hedgehog reconciles the declared state to the switches.

4.9. Network Profiles

- **Minimal (four networks):** Converged SO-C+Storage, Scale-out, In-band management, Out-of-band management (OoB unmanaged by Hedgehog).
- **Full (five networks):** Same as minimal but with separate SO-C and Storage networks.

4.10. Operations Alignment

- **Day 0:** ONIE-based discovery and Zero-Touch Provisioning (ZTP) enroll switches without manual CLI configuration.
- **Day 1:** Apply intent via Kubernetes CRDs using `kubectl apply` or `hhfctl`; validate using connectivity-maps-as-code from the companion repository [15].
- **Day 2:** Continuous reconciliation maintains declared state. Monitoring and observability (telemetry export to LGTM stacks), firmware lifecycle, and configuration updates follow the same Kubernetes-native declarative model. See the Lifecycle and Automation section for detailed Day 2 operations.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

5. Rail-Optimized Fabric Topology

5.1. Definition and Purpose

Rail-optimized topology is a Clos fabric in which xPU-node downlink allocation is constrained so that all NIC ports with the same xPU-affinity index (a "rail") connect to the same leaf switch, or to a contiguous set of leaves when scale requires. Purpose:

- Preserve rail locality for collective patterns and simplify troubleshooting
- Reduce cross-spine traffic for rail-local communications
- Provide deterministic wiring and labeling

Rail-optimized wiring materially reduces spine utilization by keeping scale-out exchanges leaf-local when tenants/workloads are placed within a common first-hop rail domain (servers whose rail-n NICs land on the same leaf). Because uplink and spine congestion is the primary source of performance degradation in large AI clusters, this locality yields measurable benefits in tail latency and job throughput across compositions of any size. Rail-optimized is a wiring allocation constraint; switch counts and bandwidth ratios are unchanged versus standard Clos (the fabric remains non-blocking).

5.2. Critical Classification: Switch Counts Unchanged

Rail-optimized does NOT change the Clos stage structure, switch counts, or port budgets compared to single-homed Clos. The difference is purely in downlink allocation to leaves.

Metric	Single-homed Clos	Rail-Optimized
Leaf count (64 servers)	8	8
Spine count (64 servers)	8	8
Downlinks per leaf	64	64
Uplinks per leaf	64	64
Oversubscription	Same	Same
Difference	Intra-group cross-rail traffic remains local, DAC/AEC conducive cabling pattern.	Larger 1st-hop rail domain size

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

5.3. Rail Concept (NIC-GPU Affinity)

Why NICs have GPU affinity: Inside each xPU-node, the xPUs and scale-out NICs communicate over a PCIe-switched internal network. Each xPU slot has a shortest path across the PCIe topology to a specific NIC slot, making that NIC the preferred path for that xPU's scale-out traffic under normal circumstances. In simplified terms: the xPU in slot 1 prefers to use the NIC in slot 1, the xPU in slot 2 prefers to use the NIC in slot 2, and so on through all eight xPU/NIC combinations.

What a rail is: This xPU-to-NIC affinity is the basis for the rail concept. Each xPU/NIC combination on a server represents one rail. In the 8-xPU-per-server form factor targeted by this RA, each xPU-node has 8 rails, numbered 1 through 8. Rail-1 refers to NIC-1 on every xPU-node; rail-2 refers to NIC-2; and so on through rail-8.

At a small scale, all of a rail's NICs may connect to a single leaf switch — but this is not always the case. With the DS5000 at 1×400GbE and 64 downlinks per leaf, a single leaf can serve at most 64 rail-n NICs (from 64 xPU-nodes). Beyond that scale, a rail's NICs span multiple leaves, and the cluster naturally divides into multiple first-hop rail domains within each rail. For example, in a 128-xPU-node cluster, rail-1 has 128 NICs spread across two leaves — forming two first-hop rail domains of 64 xPU-nodes each. The 8-NIC xPU-node form factor means each server can connect to at most 8 leaf switches, and common cabling patterns naturally produce groupings of xPU-nodes with complete rail-n first-hop connectivity within each group. Consider a cluster of 1024 nodes and 8192 xPUs with a rail-optimized topology. Using a 51T switch such as the Celestica DS5000, a single leaf switch can support up to 64 server NIC connections, making the largest 1st hop rail domain 64 nodes. A 1024 node rail-optimized network would be composed of 16 64 node 1st hop rail domains.

How "rail-n" is used in this paper: When this RA states "all rail-n NICs connect to the same leaf switch," it means that for a particular grouping of xPU-nodes, all of their rail-1 NICs connect to the same leaf, all of their rail-2 NICs connect to the same leaf, and so on for each rail. The

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

leaf that serves rail-1 may or may not be the same leaf that serves rail-2 — that depends on the rail domain sizing (see Wiring Allocation at Different Scale below).

5.4. Wiring Allocation at Different Scale

Baseline assumptions: 8 NICs per server (one per GPU); DS5000 provides 64×400G downlinks per leaf.

Formulas:

- $Total\ leaves = (Servers \times 8\ NICs) \div 64 = Servers \div 8\ NICs\ per\ rail = Servers$

Mapping at common scales:

Servers	NICs/Rail	Rails/Leaf	Total Leaves	Mapping Pattern
8	8	8	1	All 8 rails on Leaf 1
16	16	4	2	Rails 1–4→L1, Rails 5–8→L2
32	32	2	4	Rails 1–2→L1, 3–4→L2, 5–6→L3, 7–8→L4
64	64	1	8	Rail 1→L1, Rail 2→L2, ..., Rail 8→L8

Sweet spot (64 servers): Each rail has 64 NICs, matching DS5000's 64 downlinks. Each rail maps one-to-one to a leaf.

Scales > 64 servers: Each rail spans multiple leaves. For 128 servers, each rail has 128 NICs and maps to two leaves (8 rails × 2 leaves = 16 leaves total).

5.5. Dual-Plane: Doubling the First-Hop Rail Domain

The wiring allocation table above assumes single-plane designs with 1×400GbE per scale-out NIC. Dual-plane designs fundamentally change the math by using 2 ports per NIC, in this example 2x200G, providing the same scale-out bandwidth. This has a direct impact on first-hop rail domain sizing that is arguably dual-plane's most significant benefit.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

With single-plane at 1×400GbE, the DS5000 provides 64×400G downlinks per leaf. At most 64 rail-n NICs can terminate on one leaf, capping the first-hop rail domain at 64 xPU-nodes.

With dual plane at 2×200GbE, the same DS5000 provides 128×200G downlinks per leaf (via 4×200G breakout). Each xPU-node's rail-n NIC has two 200G ports: one connects to leaf-plane-A and the other to leaf-plane-B. Because each port is 200G rather than 400G, the leaf can serve 128 rail-n NICs — doubling the first-hop rail domain to 128 xPU-nodes, with no additional switches required.

Configuration	Downlinks per leaf	Max rail-n NICs per leaf	Max first-hop rail domain
Single-plane (1×400G)	64×400G	64	64 xPU-nodes (512 xPUs)
Dual-plane (2×200G)	128×200G	128	128 xPU-nodes (1,024 xPUs)
Dual-plane (2×400G)	64×400G per plane	64 per plane	64 xPU-nodes per plane (512 xPUs), but with 2× bandwidth

The 2×200G dual-plane configuration is especially compelling because it achieves the same aggregate per-NIC bandwidth as single-plane 1×400G (2×200G = 400G total) while doubling the first-hop rail domain and adding plane-level fault isolation. Modern collective communication libraries such as NCCL have mature support for dual-plane topologies, allowing frameworks to exploit both planes for bandwidth while preserving the rail locality advantages of the larger domain.

For operators choosing between single-plane and dual-plane, this domain-doubling effect should be weighed alongside the more commonly cited benefits of redundancy and maintenance isolation. In many deployments, the ability to keep twice as many xPU-nodes

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

within a single first-hop rail domain - significantly reducing spine traffic for intra-rail collectives - may be the most operationally impactful advantage.

The wiring allocation table for dual-plane at 2×200G per NIC:

xPU-Nodes	NICs/Rail	Rails/Leaf	Total Leaves (per plane)	Mapping Pattern
8	8	16	1	All 8 rails on 1 leaf per plane
16	16	8	1	All 8 rails on 1 leaf per plane
32	32	4	1	All 8 rails on 1 leaf per plane
64	64	2	1	2 rails per leaf per plane
128	128	1	1	1 rail per leaf per plane (sweet spot)

The dual-plane sweet spot is 128 xPU-nodes — exactly double the single-plane sweet spot — where each rail maps one-to-one to a leaf on each plane.

5.6. Deterministic Mapping Scheme

For automated cabling manifests:

```
rails_per_leaf = 64 / S
leaf_for_rail(r) = ceil(r / rails_per_leaf)
```

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

For $S > 64$ xPU-nodes (multiple leaves per rail):

```
leaves_per_rail = S / 64  
leaf_range_for_rail(r) = [(r-1) * leaves_per_rail + 1, r * leaves_per_rail]
```

Example (32 xPU-nodes):

```
rails_per_leaf = 64 / 32 = 2
```

Rail labeling at the leaf: Label each leaf with its rail assignments (for example, "be-leaf-01: rails 1,2" for a 32-xPU-node deployment) to simplify cabling verification and troubleshooting. Composition variants in the companion repository [15] include these labels in their connectivity maps.

5.7. Single-homed Clos vs Rail-Optimized (Summary)

Aspect	Single-Homed Clos	Rail-Optimized
OPG-M alignment	Single-homed cabling pattern [1]	Rail-optimized cabling pattern [1]
Leaf count	xPU-Nodes / 8	xPU-Nodes / 8
Spine count	Leaves / 2 (minimum non-blocking)	Leaves / 2 (minimum non-blocking)
Port utilization	Same	Same
Bandwidth ratio	Non-blocking (1:1)	Non-blocking (1:1)
Wiring allocation	All 8 NICs per xPU-node to same leaf	NICs distributed by rail affinity
First-hop domain	8 xPU-nodes per leaf; all traffic (intra-rail + cross-rail) is leaf-local	Varies by rail domain sizing; intra-rail is leaf-local; cross-rail may traverse spine
Best suited for	Workloads with high cross-rail traffic volumes; environments where first-hop domain locality (all traffic leaf-local) outweighs rail-domain scale	Workloads with predominantly intra-rail traffic patterns; rail locality benefits hold regardless of how many tenants or jobs share the domain
Cabling complexity	Lower (all NICs to same leaf)	Higher (must track rail-to-leaf assignments)

The companion repository [15] provides fully specified variants for both single-homed Clos and rail-optimized wiring at every OPG size (OPG-64 through OPG-512) and XOC tier (XOC-256 through XOC-1024).

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

5.8. Cabling Plant Considerations

The choice between single-homed Clos and rail-optimized wiring has significant implications for the physical cabling plant — implications that grow with cluster density.

Rail-optimized cabling topology: In a typical rail-optimized OPG-512 with an 8-leaf rail domain, each of the 64 xPU-nodes connects its NICs across 8 different leaf switches. This produces an end-of-row cabling pattern where the large majority of server-to-leaf connections span multiple racks, requiring fiber optics for most links and generally precluding the use of Direct Attach Cables (DACs) or Active Electrical Cables (AECs). For air-cooled deployments, rack density is often low enough (~2 xPU-nodes per rack) that few connections could use DACs for server to leaf connections in any case. However, as the server-to-leaf connections in rail optimized typically require fiber cable distance, even short reach optics for most cluster sizes could allow the leaf switches to be placed in proximity to spines, thereby allowing the leaf-to-spine connections to leverage DAC/AEC cables. Given these networks are provisioned 50%uplink/50%downlink, the tradeoff in cabling complexity between either pattern is roughly equivalent.

Single-homed Clos cabling topology: In the single-homed Clos pattern, all 8 NICs from each xPU-node connect to the same leaf switch. For liquid-cooled deployments at 8 xPU-nodes per rack, this creates an exact match: $8 \text{ xPU-nodes} \times 8 \text{ NICs} = 64 \text{ ports}$, which is precisely the downlink capacity of a single DS5000 leaf. This enables a true top-of-rack cabling pattern where one leaf switch serves one rack, and all server-to-leaf connections are short enough to use DACs or AECs. The result is a single-rack building block containing 64 xPUs and one leaf switch, where all traffic within the rack's first-hop domain is leaf-local.

Why DACs and AECs matter: DACs and AECs receive attention primarily for their lower acquisition cost compared to optical transceivers, but the operational advantages are often more consequential:

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

- **Reliability:** Passive copper connections eliminate the optical transceiver as a failure point, measurably reducing link failure rates at scale.
- **Power and cooling:** DACs and AECs consume substantially less power and produce less heat than optical modules — a meaningful consideration when multiplied across thousands of links.
- **Operational simplicity:** Fewer optics means fewer spares to stock, fewer firmware versions to manage, and fewer failure modes to troubleshoot.

Aligning architecture to traffic patterns: The key architectural decision is not "big jobs vs. small jobs" but rather the expected traffic profile - specifically, the ratio of intra-rail to cross-rail traffic. Rail-optimized designs reduce spine utilization for any mix of tenants and job sizes as long as the traffic is predominantly intra-rail; the benefit holds regardless of how many tenants share a first-hop rail domain. Larger rail domains also offer a practical bin-packing advantage: it is easier to allocate a varied mix of job sizes (for example, a service mix of 2-node, 4-node, 8-node, and 16-node tenancy allocations) into 64-node domains than into 8-node domains.

Assuming tenancy or workload scheduling can be optimized for it, Single-homed Clos becomes the stronger choice specifically when cross-rail traffic is a dominant fraction of the workload - because its first-hop domains keep all traffic (intra-rail and cross-rail alike) leaf-local for groups of 8 xPU-nodes.

Understanding the network traffic profile is essential for architecture selection. The large majority of well-known networking challenges in scaled AI clusters — tail latency, incast, congestion-induced retransmissions — trace back to congested uplinks and spines. There can be substantial benefit in aligning network architectures to workload demands, and organizations that analyze their traffic patterns can increasingly optimize infrastructure design as they scale. The right topology is the one that matches the deployment's traffic characteristics, not necessarily the one with the largest or smallest domain.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

6. Single-Plane and Dual-Plane Fabric Options

6.1. Overview

Single-Plane: Utilizes a single, unified scale-out Clos fabric for all network traffic. Each GPU-affinitized NIC connects via one high-speed port (such as ConnectX-7, 1x400G per GPU). Total: 8 scale-out ports per xPU-node.

Dual-Plane: Consists of two isolated, redundant scale-out Clos fabrics (Plane A and Plane B). This design improves redundancy and allows maintenance without downtime. Each GPU-affinitized NIC has two independent ports (such as ConnectX-8, 2x400G per GPU), one connecting to each plane. Total: 16 scale-out ports per xPU-node.

6.2. Dependency: Server NIC Port Count

NIC port count determines plane feasibility:

- **1 port per GPU-affinitized NIC** (such as ConnectX-7@1x400G): Single-plane only
- **2 ports per GPU-affinitized NIC** (such as ConnectX-7@2x200G or ConnectX-8@2x400G): Dual-plane feasible (one port to each plane)

6.3. Single-Plane Architecture

Aspect	Single-Plane
Fabric count	1
Ports per GPU-NIC	1
Total fabric bandwidth	100%
Redundancy model	Within-fabric (ECMP)
Plane failure impact	Total fabric loss
Hardware requirement	1x400G (ConnectX-7 or equivalent) per NIC

6.4. Dual-Plane Architecture

Aspect	Dual-Plane
Fabric count	2 (Plane A, Plane B)
Ports per GPU-NIC	2 (one per plane)
Total fabric bandwidth	2x single-plane (one plane per port)
Redundancy model	Plane isolation + within-plane ECMP
Plane failure impact	50% bandwidth; workloads continue
Hardware requirement	2x200G (ConnectX-7 or equivalent) per NIC

6.5. Dual-Plane Considerations

First-hop rail domain doubling: As detailed in the Rail-Optimized Fabric Topology section, dual-plane designs at 2x200G double the first-hop rail domain size from 64 to 128 xPU-nodes compared to single-plane at 1x400G. This is often the most operationally impactful advantage of dual-plane.

Plane-level staged rollouts: Changes can be applied to Plane A first, validated, then applied to Plane B. This reduces risk for configuration changes and software upgrades.

Plane maintenance is still service-impacting: Taking a plane offline reduces aggregate bandwidth by 50%. Tenants must be notified and may need to prepare - workloads that typically

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

consume more than 50% of available bandwidth will experience degradation. Dual-plane maintenance is less disruptive than a full fabric outage but is not transparent to users.

Single-plane fabrics already provide substantial resilience: Hedgehog maintains clean control-plane and data-plane separation - the controller can go completely offline and the only impact is the inability to make configuration changes; the data plane continues operating as configured. Each switch runs its own local SONiC control plane sufficient for independent operation. Hedgehog supports multiple multi-homing technologies (ESLAG, HostBGP) and strives for every fabric-attached device to be multi-homed. Switch upgrades can be performed in a rolling order so that multi-homed devices maintain connectivity throughout. The large majority of Hedgehog controller upgrades are not service-impacting; on rare occasions when they are, the interruption is brief and affects only the ability to make configuration changes.

6.6. Trade-offs and Selection Criteria

Factor	Single-Plane	Dual-Plane
First-hop rail domain (1×400G vs 2×200G)	Up to 64 xPU-nodes	Up to 128 xPU-nodes (doubled)
Hardware cost	Comparable — dual-plane uses 2×200G ports on the same NIC; switch and leaf counts are driven by xPU count, not plane count	Comparable — same number of switches for the same domain size
Cabling	One set of server-to-leaf connections	Two sets of server-to-leaf connections (one per plane); more cable runs but same leaf count per plane
Fault isolation	Within-fabric resilience via ECMP, multi-homing, rolling upgrades, and controller/data-plane separation	Adds plane-level isolation on top of within-fabric resilience
Maintenance impact	Rolling switch upgrades with minimal disruption for multi-homed devices; controller upgrades rarely service-impacting	Plane-by-plane maintenance possible but still reduces bandwidth by 50%; tenant notification required
Staged rollouts	Changes applied fabric-wide (with canary patterns available)	Changes applied plane-by-plane with validation between
NIC requirements	1 port per NIC	2 ports per NIC

Date: April 2026


 This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

The primary drivers for dual-plane selection are the doubled first-hop rail domain size and the ability to perform plane-level staged rollouts. The primary driver for single-plane is simplicity - one fabric to manage - and the inherent resilience of Hedgehog's architecture already provides substantial fault tolerance without plane-level separation.

Canonical examples from the composition repository:

Example	Topology	Scale-out NIC	Planes	Tokenized Name
OPG-64 entry	Rail-optimized	CX7 1x400G	Single	<code>clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g</code>
OPG-256 dual	Dual-plane	CX8 2x400G	Dual	<code>dual-plane--cx8-2x400g--bf3-2x200g--2p--storage-conv-2x200g</code>
XOC-512 composed	Rail-optimized	CX7 1x400G	Single	<code>clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g</code>

Decode: `<topology>--<scale-out-NIC>--<frontend-NIC>--[planes]--<storage>`

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

6.7. Rack-Level Wiring by Density and Cooling

The network fabric design is independent of cooling modality - the same topology, control plane, and overlays apply to both air-cooled and liquid-cooled deployments.

Cable lengths: Composition variants provide default cable lengths suitable for common rack and row geometries. Operators should adjust cable lengths to match exact cable run distances for their deployment once site layout is finalized.

6.8. Wiring and Operational Considerations

- **Physical separation:** Prefer separate routes/patching for planes to reduce correlated failures.
- **Management model:** Treat planes as separate Hedgehog-managed fabrics or separate namespaces; monitor per-plane health and alerts.
- **Capacity planning:** Ensure base workload remains viable at ~50% bandwidth during single-plane maintenance or failure.

7. Overlay and Policy

This section describes how logical overlays and policies are realized using the Hedgehog VPC abstraction, with EVPN for control plane and VXLAN for data plane. It also outlines RoCE-oriented QoS, including one-parameter enablement to standardize lossless class configuration and hashing suitable for AI workloads.

7.1. Overlay Model (VPC → VRF/VNI)

- **Abstractions:**
 - VPC maps to a VRF (tenant routing context).
 - VPC subnets map to VNIs (one per segment), with symmetric IRB for routing.
 - Route Distinguishers (RDs) and Route Targets (RTs) are allocated deterministically per VPC and subnet.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

- **Realization:**
 - EVPN Type-2 routes advertise MAC/IP reachability for bridged endpoints.
 - EVPN Type-5 routes advertise IP prefixes for routed segments and external connectivity.
 - BGP distributes EVPN routes across the fabric; SONiC applies data-plane state.
- **Broadcom Tomahawk5 (TH5) optimization:**
 - On Broadcom TH5 leaves, a pure L3VNI model is required because TH5 has chipset limitations for L2VNI and Broadcom Enterprise SONiC does not support L2VNI on this platform. All overlay traffic on DS5000 switches is therefore Layer 3.

7.2. Virtual Private Cloud (VPC) API Resources and Policy Control

- **Resources:**
 - *VPC* defines the VRF and default import/export policy.
 - *VPCAttachment* binds server interfaces to VPCs with tagging.
 - *External*, *ExternalAttachment*, *ExternalPeering* define north–south connectivity and BGP peers.
 - *VPCPeering* optionally enables routed connectivity between VRFs with policy controls.
- **Policy:**
 - Import/export targets per VPC enforce isolation by default and allow scoped connectivity (for example, selective inter-VPC peering).
 - Route policies control redistribution to externals (for example, advertise only Type-5 routes to upstream peers).
 - Default policies are safe and minimal; tuned profiles can be provided for specific deployment needs.

7.3. Per-Network Considerations

- **Scale-out fabric:** Prioritize throughput and predictable latency; L3VNI is required on TH5-based leaves (DS5000) due to chipset and SONiC distribution limitations for L2VNI; ECMP hashing optimized for RDMA. Multi-homing is not required for NCCL/RCCL traffic in most deployments because each xPU-node NIC connects to a single leaf. In rail-optimized deployments, rail-local traffic remains at the leaf; cross-rail traffic traverses spines.
- **Storage Fabric:** Storage Fabric: Balance throughput and resilience; multi-homing is recommended for storage resilience. HostBGP provides L3 multi-homing with automatic ECMP failover and no L2 protocol overhead; ESlAG provides L2 port-channel semantics where required. Conservative route policies toward external storage networks.
- **SO-C / Storage fabric (converged in Minimal profile):** Front-end connectivity uses HostBGP for L3 multi-homing across redundant leaves (see baseline assumptions). Lower east-west volume; focus on north-south policy and NAT/firewall integration at border.
- **In-band network:** Protect telemetry/control; rate-limit and prioritize; accordingly, isolate from tenant overlays.
- **Scale-out CPU fabric (SO-C, separate in Full profile):** Treat as a dedicated network with tuned east-west policies when separated from the storage fabric.

7.4. RoCE and QoS Approach

- **Objectives:** Provide lossless treatment for RDMA traffic classes and ECMP-friendly flow distribution; avoid complex per-device tuning.
- **One-parameter enablement:** A single configuration toggle enables the lossless queue for the RDMA class, PFC thresholds, ECN marking thresholds, and appropriate ECMP hashing. Hosts select the RDMA class via DSCP marking.
- **Components:**

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

- PFC (802.1Qbb) enabled for the RDMA class.
 - ECN marking tuned for congestion signaling suitable for collective operations.
 - DSCP mapping selects queues consistently across devices.
 - Queue Pair Number (QPN)-aware hashing improves load distribution for RDMA flows.
- **Profiles:** Provide safe defaults per size tier; expose parameters for tuning based on workload observation. Document operational limits.

7.5. Host and NIC Performance Tuning

- Fabric-level QoS provides the network foundation for lossless RDMA transport, but achieving optimal end-to-end training performance also requires careful host OS and NIC configuration. Host-side tuning — including NIC firmware settings, interrupt affinity, memory registration policies, and kernel parameters — has a material impact on collective operation throughput and tail latency. FarmGPU developed a highly optimized host and NIC tuning configuration during their XOC-512 B200 deployment that delivers exceptional results as confirmed by SemiAnalysis performance analysis. FarmGPU has generously shared detailed information on their NIC and host OS performance tuning approach, which is provided in the companion repository [15]. Operators are encouraged to use these tuning recommendations as a starting point and adapt them based on workload-specific observation.

7.6. Border Connectivity

- Border leaves (for example, Celestica DS3000 switch) connect tenant VRFs to upstream networks using *Externa* resources and EVPN Type-5 redistribution as appropriate.
- NAT/PAT and firewalling are provided by a separate, distributed Hedgehog gateway on x86 servers (optional component), managed via Kubernetes-native APIs and scaled by adding servers.
- Border policies are explicit: advertise only required prefixes; maintain tenant isolation by default; use per-VRF policies for controlled exposure.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

8. Lifecycle and Automation

This section describes the operational lifecycle for deploying and managing XOC-N aligned AI fabrics using the Hedgehog. The framework follows Day 0 (initial deployment), Day 1 (intent application), and Day 2 (operations). The goal is to make the process concrete and reproducible using open interfaces and declarative artifacts consistent with OCP expectations.

8.1. Day 0: Initial Deployment

Discovery via ONIE

Switches ship with ONIE (Open Network Install Environment), an open-source boot loader for bare-metal switches originating from the Open Compute Project. On power-on, ONIE enters discovery mode and requests network configuration via DHCP. In this RA, DHCP options provide the location of the Hedgehog controller/installer so that switches can enroll automatically without manual configuration. ONIE attribution: ONIE is an OCP project; Hedgehog works with ONIE to automate switch onboarding.

Zero-Touch Provisioning (ZTP) Workflow

The Zero-Touch Provisioning workflow removes the need for device-level CLI and templates. The sequence is:

- Switch boots into ONIE and obtains network parameters via DHCP.
- DHCP response includes the Hedgehog controller/installer endpoint.
- The switch downloads and installs the Hedgehog agent and the base SONiC image appropriate for its platform.
- The agent registers with the Hedgehog controller and presents its identity and platform characteristics.
- The controller assigns switch identity, Autonomous System Number (ASN) and management parameters, and binds a switch profile that encodes platform-specific capabilities (for example, port layout, speed/FEC, breakout, RoCE capabilities).

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

- The agent configures SONiC accordingly and establishes management channels (for example, gNMI for configuration/telemetry).
- The switch appears as a managed resource and is ready for intent application.

Key properties:

- No manual CLI on switches; identity and keys are provisioned automatically.
- Switch profiles abstract hardware specifics, enabling multi-vendor operation with the same logical design.
- The outcome is a SONiC-based switch that acts as a data plane element controlled by the Hedgehog controller, ready to participate in the fabric.

Initial Verification

Upon successful registration, the controller confirms switch reachability and basic health. Minimal checks include agent heartbeat, gNMI readiness, and platform profile application. At this point, operators can list inventory and status using standard Kubernetes tools (for example, *kubectl get* against CRDs) or the Hedgehog CLI. The fabric is now ready for Day 1 intent.

8.1. Day 1: Configuration

Intent Application (Declarative CRDs)

Configuration is expressed as Kubernetes Custom Resource Definitions (CRDs) and applied via *kubectl* or the Hedgehog CLI. Users do not write device-level configuration for switches. Two API groups are central to Day 1:

- Wiring API (physical topology):
 - *Switch* resources describe fabric switches (role, profile, ASN, addressing).
 - *Server* resources describe attached servers.
 - *Connection* resources define physical/logical connectivity (server↔leaf, leaf↔spine, external links) and link aggregation types (unbundled, bundled, ESRAG, fabric links).
- VPC API (logical overlays):

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

- *VPC* resources represent tenant VRFs; subnets map to VNIs.
- *VPCAttachment* binds server interfaces into VPCs.
- *External/ExternalAttachment/ExternalPeering* define north-south connectivity and BGP peering.
- *VPCPeering* enables routed connectivity between VPCs when required.

Operators store these artifacts as versioned documents (YAML) and apply them in logical units (for example, per-tier or per-tenant). The controller computes the desired state and the agents render the corresponding SONiC configuration on the switches. This process is compatible with GitOps: the CRDs serve as the source of truth, supporting review, change control, and rollback.

Validation and Pre-Deployment Checks

Before resources are accepted, schema validation and admission webhooks perform structural and semantic checks (for example, required fields, role/profile compatibility, connection endpoint validity). A connectivity-map generator derives expected switch-to-switch and server-to-leaf mappings and can be used to validate cabling plans and port/NIC mapping prior to physical turn-up. These checks catch errors early and reduce the risk of partial or inconsistent deployments.

Rollout and Change Patterns

Changes can be introduced incrementally to reduce risk:

- **Canary rollout:** apply a new topology or overlay change to a subset of racks or leaves, observe telemetry, then expand.
- **Sequenced updates:** the controller orchestrates application order to maintain reachability and policy invariants.
- **Rollback:** revert CRD changes in git; the controller reconciles to the prior state.
- **Safe operations:** drain traffic from affected attachments where appropriate (for example, when changing LAG types) to minimize disruption.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

Because configuration is declarative and continuously reconciled, Day 1 is repeatable and auditable. Operators can introduce tenants, attach hosts, and adjust external connectivity through CRDs without device-specific procedures.

8.2. Day 2: Operations

Observability

Telemetry collection uses Alloy, Grafana's OpenTelemetry collector, to gather metrics and logs from switches and agents. Data sources include gNMI streaming telemetry and process metrics from the Hedgehog controller and agents. Export targets follow the common LGTM stack:

- Loki: log aggregation.
- Grafana: dashboards and visualization.
- Tempo: distributed tracing.
- Mimir: metrics storage (Prometheus-compatible).

Standard interfaces are used end-to-end (Kubernetes APIs, gNMI on devices, Prometheus exposition, and Redfish/OpenBMC for hardware inventory where applicable). This enables integration with existing enterprise tooling and simplifies operational runbooks across environments.

Continuous Reconciliation and Drift Management

The Hedgehog controller continuously compares desired state (Kubernetes CRDs) with actual state observed from switches. Deviations (for example, accidental configuration drift, link role changes, or component replacement) are detected and corrected by reconciliation. When operators commit changes to Kubernetes CRDs, reconciliation computes the minimal delta and applies it safely through the agents. If the Hedgehog controller is temporarily unavailable, switches continue forwarding with the last applied SONiC configuration; reconciliation resumes when the controller returns.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

Failure Handling

Hedgehog agents report health and status to the Hedgehog controller, including switch reachability and key component metrics. The controller flags anomalies (for example, a leaf becomes unreachable or loses upstream links) and provides context to observability pipelines for alerting and triage. Recovery procedures include replacing failed hardware and reassigning identity so that the same logical configuration re-applies, or isolating impacted attachments while maintaining fabric integrity. Because intent is authoritative, replacement workflows are deterministic and largely hands-off beyond physical swap.

Maintenance Operations

Common Day-2 operations are coordinated through the controller using declarative inputs:

- Switch OS upgrades: roll SONiC image updates in a controlled sequence aligned to topologies and maintenance windows.
- Agent updates: update Hedgehog agent versions centrally and verify health gates before proceeding cluster-wide.
- Configuration changes: modify CRDs for overlays, policies, or topology; controller orchestrates safe application.
- Capacity expansion: add new switches, apply Switch and Connection resources, and observe reconciliation; add spines/leaves per size-tier templates.
- Hardware replacement: replace a failed switch, associate prior identity/profile, and allow reconciliation to repopulate configuration.

Day-2 processes emphasize low-risk, observable changes backed by declarative state and automated reconciliation, aligning with reliability goals for training workloads.

Multi-OPG Operational Considerations

When clusters span multiple OPG-M building blocks, stage operational workflows per OPG. For example, roll out upgrades one leaf group at a time, validate health/traffic, then proceed to the

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

next OPG. Treat OPG boundaries as failure domains at the leaf layer and plan change windows accordingly.

Dual-Plane Operational Considerations (if applicable)

In dual-plane configurations, perform plane-by-plane maintenance: upgrade Plane A while Plane B continues to carry traffic (at approximately 50% aggregate bandwidth), validate, then upgrade Plane B. Use staged rollouts: apply changes on Plane A first, observe SLOs and health, then apply to Plane B. Monitor per-plane telemetry and alerts to distinguish issues.

9. Validation Plan

This section outlines the network-centric validation methodology for the AI fabric. It focuses on fabric topology/overlay correctness and operational behavior without claiming in-house, full-scale reproduction. Field deployments and external reports inform expectations; lab-scale emulation and logical validations provide repeatability. Compute, storage, and facility validation are out of scope.

9.1. Objective and Scope

- Verify logical correctness of topology and overlay expression (Wiring/VPC CRDs, connectivity-maps-as-code).
- Validate operational workflows: ZTP, intent application, reconciliation behavior, and failure handling.
- Exercise representative traffic patterns (synthetic and NCCL/RCCL-like) to observe latency and convergence behaviors.
- Confirm observability coverage (telemetry signals, dashboards, alerts) sufficient for Day-2 operations.
- Verify tenant placement within first-hop rail domains for targeted workloads (scheduling audit).

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

- Monitor spine link utilization before and after placement alignment; expect measurable reductions in scale-out spine traffic.
- Track tail latency (p95/p99) improvements correlated with rail-domain-local placement.
- These scheduling and placement checks complement the Scheduling and Tenancy Guidance in Architecture Overview.

9.2. Methods

- **Logical validation:**
 - Schema validation and admission checks for CRDs.
 - Connectivity-maps-as-code to ensure port/NIC mapping, link roles, and separation between networks adhere to design rules.
 - Policy checks (for example, VRF isolation, import/export targets, external redistribution constraints).
- **Lab-scale emulation:**
 - Emulate example tiers (for example, OPG-64/OPG-128 building blocks composing to XOC-64/XOC-128) with SONiC-based devices and representative host endpoints.
 - Generate synthetic traffic patterns and NCCL/RCCL-like workloads to observe tail-latency and throughput under load.
 - Perform failure drills: link/node failure, agent/controller restarts, maintenance events; confirm continuity and reconciliation behavior.

Hedgehog provides a virtual lab (VLAB) environment that runs the same Hedgehog controller and Broadcom Enterprise SONiC (via virtual SONiC switches) used in production deployments. Operators can apply the wiring CRDs from any composition in the companion repository[15] to instantiate that topology in VLAB — the same CRDs that would configure a physical fabric are used unchanged in the virtual environment. This enables operators to validate topology, overlay, and policy configuration, practice the full ZTP-to-Day-2 lifecycle, and gain hands-on experience operating the Hedgehog solution before committing to physical infrastructure. The companion

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

repository includes a dedicated XOC-Virtual composition designed for this purpose: it provides virtual servers with a comparable network configuration so operators can experience the complete process of applying composition assets to a fabric, from initial switch enrollment through tenant overlay configuration. VLAB uses SONiC virtual switches that provide broad command coverage (~95%) without requiring physical switch hardware. Virtual switch port counts may differ from physical platforms, breakout emulation is not supported, and performance and scale are constrained by the virtualized environment. VLAB is well-suited for validating configuration correctness, CRD workflows, reconciliation behavior, failure drills, and operator training — it is not intended for performance benchmarking.

- **Observability:**
 - Use Alloy→LGTM to collect logs, metrics, and traces; confirm coverage of key device and control-plane signals.
 - Establish dashboards for health, capacity, and change monitoring; validate alerts for material failure modes.

9.3. Success Criteria

- **Topology correctness:** No violations detected by connectivity-map checks; accurate labeling/Bill Of Material (BoM) alignment.
- **Overlay correctness:** Expected EVPN routes present (Type-2/Type-5) and policy boundaries enforced.
- **ZTP and intent:** Switches enroll and apply intent with no manual CLI; drift is reconciled within acceptable time windows.
- **Performance signals:** Observed traffic patterns exhibit predictable throughput and latency for the chosen profiles at lab scale; deviations are explainable.
- **Failure handling:** Expected behaviors observed during link/node faults and upgrades; recovery is deterministic and low-risk.

9.4. Limitation and External References

- **Scale limitations:** This plan does not claim in-house validation at XOC-1k scale; results are extrapolated from lab-scale emulation and field references.
- **External validation:** FarmGPU, a neocloud operator and OCP community leader, deployed and currently operates an XOC-512 B200 cluster using this Reference Architecture. SemiAnalysis independently analyzed FarmGPU's xPU infrastructure and reported that the network fabric outperformed a number of other solutions tested [12][13]. FarmGPU's public blog series documents their deployment experience, including a rapid bring-up cycle [14]. Additional operators deploy compositions from this RA at various scale tiers. These field references inform the performance and operational expectations described throughout this document.
- **Hardware variability:** Vendor-specific differences are abstracted by switch profiles; validation confirms profile coverage but does not replace platform certification.

9.5. Packaging and Reproducibility

- **Artifacts:** Provide sample CRDs, connectivity-map rules, validation scripts, and runbooks to enable independent verification by adopters.
- **Process:** Document step-by-step procedures for Day 0/1/2 tests and failure drills; include pass/fail gates and rollback guidance.
- **GitOps:** Store validation inputs/outputs alongside intent artifacts for auditability.

10. Conclusion

This RA provides a deployment-ready, network-centric design for AI fabrics using Hedgehog as the Kubernetes-native control plane and SONiC as the data-plane NOS on OCP-compliant switches such as Celestica DS5000. It specifies conceptual structure (OPG-M modular building blocks composing to XOC-N cluster architectures), overlay/policy via EVPN/VXLAN with Hedgehog VPC abstractions, RoCE-oriented QoS, and lifecycle operations grounded in open interfaces and declarative artifacts.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

By defining size-tiered templates and network profiles, the design scales via OPG-64/128/256/512 building blocks composed into XOC-64/128/256/512/1,024 tiers with additive changes and consistent operational practices. A wide range of composition variants are provided via the companion repository [15], spanning single-plane (ConnectX-7, 1x400G) and dual-plane (ConnectX-8, 2x400G) options with clear selection criteria. Rail-optimized wiring with scheduling and tenancy guidance enables operators to align workload placement with first-hop rail domains, reducing spine utilization for targeted workloads. Continuous reconciliation, connectivity-maps-as-code, and Kubernetes CRD-driven operations reduce operational risk. The open, multi-vendor approach -- realized via switch profiles -- enables adopters to preserve designs across hardware generations within the OCP ecosystem.

Composition variants with connectivity maps, BoMs, and Hedgehog Kubernetes CRDs are available in the companion repository [15] for immediate operator use. The rail-optimized wiring and single-/dual-plane options continue to apply at higher port speeds while retaining the same abstractions and operational model. The intent is to make XOC-N practices broadly adoptable with clarity, precision, and practical guidance for real deployments.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

11. Glossary

Following are key terms used in this document.

Term	Definition
Alloy	Grafana's OpenTelemetry collector used for telemetry ingestion.
ASN	The Switch BGP Autonomous System Number
BlueField-3	NVIDIA DPU (Data Processing Unit) used as the reference SO-C/Storage NIC. Provides 2x200G per xPU-node for converged Scale-out CPU fabric and Storage fabric connectivity via L3MH.
BoM	Bill of Materials that include switches, optics, cables or gateways with quantities and specifications.
CDU	Coolant Distribution Unit.
Clos	A non-blocking, multi-stage leaf-spine network topology commonly used in AI data centers. It provides high bisectional bandwidth and predictable performance.
Custom Resource Definition (CRD)	Resource describing links between endpoints; supports unbundled, LAG, ESLAG, and fabric links.
Connectivity Map	Generated mapping of physical/logical connectivity used for validation and labeling.
ConnectX	NVIDIA network interface controller family used as the reference scale-out NIC in this RA. ConnectX-7 provides 1x400G per port; ConnectX-8 provides 2x400G for dual-plane configurations.
Continuous Reconciliation	The control-plane loop executed by the Hedgehog controller that constantly monitors the network to ensure the actual state of the device matches the intended desired state.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

Term	Definition
DACs/AECs	Direct Attach Cables / Active Electrical Cables (cables types used to substitute optics).
DCIM	Data Center Infrastructure Management, a system for managing the physical infrastructure.
DHCP	Dynamic Host Configuration Protocol, a network protocol used by the switches during Zero-Touch Provisioning (ZTP) to obtain initial network configuration and the location of the Hedgehog controller.
DSCP	Differentiated Services Code Point; the six-bit field in the IP header used for classifying and managing network traffic, allowing hosts to select Quality of Service (QoS) class.
Dual-plane	A network design that uses two isolated and redundant scale-out fabrics (Plane A and Plane B). Each xPU-affinitized NIC has two ports; with one connecting to each separate plane for high availability and maintenance isolation.
ECN	Explicit Congestion Notification; a mechanism where network devices can signal impending network congestion to end-hosts by marking the IP header, allowing the endpoints to reduce their transmission rate proactively.
ESLAG	Ethernet Segment LAG (EVPN-based multi-homing with DF election).
EVPN	BGP control plane for VXLAN (Type-2 MAC/IP routes, Type-5 IP prefixes).
External	Logical external network definition for north-south connectivity.
ExternalAttachment	Attaches a VPC to an External network.

Term	Definition
ExternalPeering	Configures BGP peering to an external network.
First-hop domain	The set of xPU-nodes whose scale-out NICs all connect to a common leaf switch in a single-homed Clos topology. All traffic between nodes within a first-hop domain — both intra-rail and cross-rail — is forwarded within the leaf switch and never traverses the spine. With the DS5000 at 1×400GbE, a first-hop domain contains 8 xPU-nodes (64 xPUs).
First-hop rail domain	The set of xPU-nodes whose rail-n NICs terminate on a common leaf (or contiguous set of leaves) in a rail-optimized topology. Intra-rail traffic within a first-hop rail domain stays leaf-local; cross-rail traffic may traverse the spine depending on rail domain sizing. Distinguished from a first-hop domain (single-homed Clos), where all traffic — not just intra-rail — is leaf-local.
FEC	Forward Error Correction; a hardware capability defined in the Switch Profile for switch ports.
GitOps	Operational model where Kubernetes CRDs are versioned in git; changes are reviewed, auditable, and reconciled.
GPU	Graphics Processing Unit; a specific example of an accelerator processing unit (xPU).
gNMI	gRPC Network Management Interface for configuration/telemetry.
Hedgehog agent	Hedgehog agent running on each device (SONiC switch) that applies state to SONiC and reports status.
Hedgehog AI Network (Hedgehog)	Kubernetes-native fabric management platform (Apache 2.0) that manages SONiC-based switches via Kubernetes CRDs and continuous reconciliation.

Date: April 2026


 This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

Term	Definition
Hedgehog controller	A core component of the Hedgehog control plane that runs within Kubernetes. It is responsible for calculating the desired state of the network and continuously reconciling the actual device state (SONiC switches) to match desired state.
Hedgehog gateway	A distributed, open-source border connectivity component that runs on x86 servers and provides policy-driven NAT/PAT and stateful firewall services for north-south traffic. Managed via Kubernetes-native APIs and scaled horizontally by adding servers.
hhfctl	Hedgehog CLI (<i>kubectf</i> plugin) for interacting with fabric Kubernetes CRDs.
HostBGP	Hedgehog's Layer 3 multi-homing feature for frontend and storage networks. A host-side FRR container establishes unnumbered eBGP sessions with redundant leaf switches; the host advertises Virtual IPs (VIPs) as /32 routes, and the fabric provides ECMP-based load balancing and failover. Enabled by setting `hostBGP: true` on the VPC subnet CRD. HostBGP subnets attach to unbundled connections only. See Hedgehog Docs [16].
IRB (symmetric)	Symmetric Integrated Routing and Bridging (IRB); a method where both Layer 2 bridging and Layer 3 routing functions for a given VLAN/VNI are performed on all edge devices (VTEPs) in the VXLAN fabric, ensuring consistent forwarding.
Kubernetes-native	An architecture built on Kubernetes primitives (Custom Resource Definitions, operators, controllers). Distinguished from "cloud-native," which refers to broader ecosystem patterns such as GitOps and containerized workloads.

Date: April 2026


 This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

Term	Definition
L2VNI	Layer 2 VXLAN Network Identifier, a 24-bit field within a VXLAN header that uniquely identifies a specific Layer 2 bridge domain, allowing it to span across a Layer 3 network
L3VNI	Layer 3 VXLAN Network Identifier used for routed overlays. TH5-based switches (such as DS5000) use L3VNI for all overlay traffic.
L3MH	Layer 3 Multi Homing, where a server connects to multiple switches to maximize bandwidth, ensure redundancy, and enable efficient load balancing.
LAG	Link Aggregation Group, a link aggregation type used to bundle multiple physical links into a single logical link.
LGTM	The LGTM observability stack, which stands for Loki (logs), Grafana (dashboards), Tempo (traces), Mimir (metrics). It provides a comprehensive solution for monitoring, logging, and tracing.
NCCL	NVIDIA Collective Communication Library, a software library that provides optimized, topology-aware, multi-GPU and multi-node communication primitives.
NIC	Network Interface Card; a server component used for network connectivity.
Non-blocking (Clos fabric)	A fabric where sufficient uplinks are reserved to provide full bisection bandwidth within the scale-out plane.
ONIE	Open Network Install Environment (OCP project); open boot loader for switch discovery and imaging.
OoB	Out of Band (used for management network)

Term	Definition
OpenBMC	Open-source baseboard management framework for platform management.
OPG-M	Open Pod Group for M xPUs; a modular cluster building block (pod group) consisting of xPU-nodes plus networking, storage, and management resources. Parameterized by total xPU count 'M'.
ORv3	Open Rack v3, a foundational rack-scale standard from the Open Compute Project.
PFC	Priority Flow Control (IEEE 802.1Qbb); link-level flow control applied selectively to specific traffic classes, enabling lossless transport for protocols like RoCE.
QPN	Queue Pair Number; used in QPN-aware hashing to improve load distribution for Remote Direct Memory Access (RDMA) flows.
Rail	The complete set of Scale-out NIC ports across all xPU-node servers that share the same GPU-affinity index. In an 8-xPU-per-server architecture, there are 8 rails (rail-1 through rail-8). Rail-1 consists of all NIC-1 ports; rail-2 consists of all NIC-2 ports; and so on. The affinity arises because each xPU slot has a shortest path across the server's internal PCIe-switched network to a specific NIC slot.
Rail domain sizing	The number of leaves assigned to a first-hop rail domain, treated as an intentional design variable rather than a fixed property of the topology. For example, an OPG-512 with 8 leaves can be configured as 1× 8-leaf domain, 2× 4-leaf domains, or 4× 2-leaf domains, each offering different trade-offs between domain size and cross-rail locality.

Date: April 2026


 This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

Term	Definition
Rail-n NIC	The NIC on a given xPU-node that belongs to rail n. For example, the rail-3 NIC is the NIC affinitized with the xPU in slot 3. When this RA states "all rail-n NICs connect to the same leaf," it means that for a grouping of xPU-nodes, every server's rail-1 NIC connects to the same leaf, every server's rail-2 NIC connects to the same leaf, and so on for each rail.
Rail-optimized	Wiring allocation constraint aligned with the rail-optimized cabling pattern in the OPG-M specification [1]. Ports of a given rail connect to the same leaf or contiguous leaf set, preserving rail locality. Does NOT change switch counts versus single-homed Clos. Note: both single-homed Clos and rail-optimized are Clos topologies; the distinction is in the server-to-leaf cabling pattern, not the topology itself.
RDMA	Remote Direct Memory Access used for lossless transport
Redfish	DMTF standard API for hardware management/inventory.
RoCE	Remote Direct Memory Access (RDMA) over Converged Ethernet; a protocol that allows direct memory access between hosts over an Ethernet network, which requires a lossless transport layer and Equal Cost Multi-Path-friendly hashing for optimal performance.
Scale-out domain	The Ethernet fabric used to interconnect multiple xPU-nodes across the cluster for large-scale communication (for example, distributed training). This domain is the primary subject of this reference architecture.
Scale-up domain	High-bandwidth interconnect within a server chassis (for example, NVLink, NVSwitch); assumed contained within the xPU-node enclosure.

Date: April 2026


 This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

Term	Definition
Server (CRD)	Resource describing a server connected to the fabric.
Single-homed Clos	Folded Clos (leaf-spine) cabling pattern aligned with the "Clos topology" pattern in the OPG-M specification [1], in which all scale-out NICs from each xPU-node connect to the same leaf switch. This creates a first-hop domain where all traffic between co-located xPU-nodes — both intra-rail and cross-rail — is leaf-local. Uses the same switch counts as rail-optimized.
Single-plane	A network design that utilizes a single, unified scale-out Clos fabric. Each xPU-affinitized NIC port on a server connects to a port on this sole fabric.
SLI	Service Level Indicator; a quantifiable measure (e.g., p99 latency, error rate, throughput). The fabric and observability stack measure SLIs.
SLO	Service Level Objective; target for an SLI (e.g., p99 < 50 ms). Operators define SLOs based on business needs.
SO-C	Scale-out Connectivity, Network providing connectivity for CPU, gateway, and ingress services. Converged with storage in both Minimal and Distributed profiles. Previously referred to as "front-end" in some contexts.
SONiC	Software for Open Networking in the Cloud; an open-source network operating system (Linux Foundation project) designed to run on various switch hardware.
Switch (CRD)	Resource describing a switch (role, profile, ASN, addressing).
Switch Profile	Hardware capability abstraction (ports, breakout, speed/FEC, platform parameters).

Date: April 2026


 This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

Term	Definition
TH5	Broadcom Tomahawk 5; 800G-class switch silicon referenced in this RA.
VLAB	Hedgehog Virtual Lab; a virtualized environment that runs the same Hedgehog controller and Broadcom Enterprise SONiC used in production, on virtual SONiC switches. Enables operators to validate configurations, practice lifecycle operations, and build familiarity without physical hardware. See also XOC-Virtual.
VNI	VXLAN Network Identifier; maps subnets/segments to overlays.
VPC	Virtual Private Cloud abstraction; maps to a VRF in the data plane.
VPC API	Hedgehog Kubernetes CRDs (vpc.githedgehog.com) defining logical overlays (<i>VPC</i> , <i>VPCAttachment</i> , <i>External</i> , <i>VPCPeering</i> , etc.).
VPCAttachment	Binds a server/port to a VPC (assigns membership and tagging).
VPCPeering	Routed connectivity between VPCs (policy-controlled).
VRF	Virtual Routing and Forwarding instance (tenant routing context).
VXLAN	Data-plane encapsulation for L2/L3 overlays over IP underlay.
Wiring API	Hedgehog Kubernetes CRDs (wiring.githedgehog.com) defining physical topology (<i>Switch</i> , <i>Server</i> , <i>Connection</i>).

Term	Definition
XOC-N	xPU Open Cluster for N xPUs; a cluster architecture composed of one or more OPG-M building blocks connected into a scale-out domain. "N" is total xPU count.
XOC-Virtual	A composition in the companion repository designed for use with Hedgehog VLAB. Provides virtual servers and a representative network topology for hands-on operator training and configuration validation without requiring physical switch hardware.
xPU	Accelerator processing unit (such as GPU, TPU, IPU) used generically in OCP contexts to refer to any specialized processor designed for acceleration.
xPU-node	A server form factor containing multiple xPUs. The high-bandwidth scale-up interconnect domain is contained entirely within the server chassis (for example, 8 xPUs per node). The definition is independent of the cooling method.
ZTP	Zero-Touch Provisioning; automated switch bring-up using ONIE and controller-driven workflows.

12. References

- [1] OPG-M System Architecture, Open Compute Project, 14 January 2026.
<https://www.opencompute.org/documents/opg-m-system-architecture-final-14-january-2026-pdf>
- [2] XOC-N System Architecture, Open Compute Project, 14 January 2026.
<https://www.opencompute.org/documents/xoc-n-system-architecture-final-14-january-2026-pdf>
- [3] Software for Open Networking in the Cloud (SONiC), Linux Foundation.
<https://sonicfoundation.dev/>
- [4] Celestica DS5000 64x800G Switch Specification, OCP Contribution.
<https://www.opencompute.org/documents/celestica-ds5000-64x800gb-switch-specification1-2-pdf> Product page: <https://cls.celestica.com/hardware-platforms/ds5000/>
- [5] Celestica DS3000 Series. <https://cls.celestica.com/hardware-platforms/ds3000/>
- [6] Celestica DS2000 Series. <https://cls.celestica.com/hardware-platforms/ds2000/>
- [7] Celestica DS1000 Ethernet Access Switch Specification, OCP Contribution.
<https://www.opencompute.org/documents/ds1000ethernetaccessswitch-docx-pdf> Product page:
<https://cls.celestica.com/hardware-platforms/ds1000/>
- [8] Open Rack v3 Specification, Open Compute Project.
<https://www.opencompute.org/documents/open-rack-v3-spec>
- [9] Hedgehog (Apache 2.0). <https://qithedgehog.com/>
- [10] ONIE -- Open Network Install Environment, Open Compute Project.
<https://opencomputeproject.github.io/onie/>

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

[11] Charles Clos., A Study of Non-Blocking Switching Networks.

<https://ieeexplore.ieee.org/document/6770468>

[12] Hedgehog, "Industry-Leading AI Network Performance."

<https://hedgehog.cloud/blog/industry-leading-ai-network-performance>

[13] FarmGPU, "B200 Cluster."

<https://blog.farmgpu.com/b200-cluster/>

[14] FarmGPU, "Building an AI Cluster: Our 17-Day Crash Course in Open Networking."

<https://blog.farmgpu.com/building-an-ai-cluster-our-17-day-crash-course-in-open-networking/>

[15] Open Cluster Designs Aligned AI Training Fabric — Companion Repository, Open Compute Project.

<https://github.com/opencomputeproject/OCP-OCDAI-training-fabric>

[16] Hedgehog Docs, Host Settings and HostBGP Container.

<https://docs.hedgehog.cloud/latest/user-guide/host-settings/>

13. License

13.1 Creative Commons - This is the suggested language.

OCP encourages participants to share their proposals, specifications and designs with the community. This is to promote openness and encourage continuous and open feedback. It is important to remember that by providing feedback for any such documents, whether in written or

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

verbal form, that the contributor or the contributor's organization grants OCP and its members irrevocable right to use this feedback for any purpose without any further obligation.

It is acknowledged that any such documentation and any ancillary materials that are provided to OCP in connection with this document, including without limitation any white papers, articles, photographs, studies, diagrams, contact information (together, "Materials") are made available under the Creative Commons Attribution-ShareAlike 4.0 International License found here: <https://creativecommons.org/licenses/by-sa/4.0/>, or any later version, and without limiting the foregoing, OCP may make the Materials available under such terms.

As a contributor to this document, all members represent that they have the authority to grant the rights and licenses herein. They further represent and warrant that the Materials do not and will not violate the copyrights or misappropriate the trade secret rights of any third party, including without limitation rights in intellectual property. The contributor(s) also represent that, to the extent the Materials include materials protected by copyright or trade secret rights that are owned or created by any third-party, they have obtained permission for its use consistent with the foregoing. They will provide OCP evidence of such permission upon OCP's request. This document and any "Materials" are published on the respective project's wiki page and are open to the public in accordance with OCP's Bylaws and IP Policy. This can be found at <http://www.opencompute.org/participate/legal-documents/>. If you have any questions, please contact OCP.

13.1. Copyright License - This is the suggested language.

© Copyright [Company Name][Year] All rights reserved.

14. About Open Compute Foundation

The Open Compute Project (OCP) brings at-scale innovations and hyperscaler best practices to all, spanning technology domains from the data center to the edge, and the technology stack from silicon, to systems, to site facilities and services. The international OCP Community is

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

made up of organizations and people from hyperscale and tier-2 cloud data center operators, communications providers, colocation providers, diverse enterprises, and technology vendors. The OCP Foundation fosters collaboration within the Community to tackle market challenges in an array of OCP Projects that create open contributions such as specifications, designs, and more. The OCP Solution Provider Program then recognizes the application of those contributions in compliant products, solutions, and data center facilities and services with designations like OCP Inspired, OCP Accepted and OCP Ready. With the tenets of openness, impact, efficiency, scale and sustainability, the OCP engages and educates thousands of engineers every year through many events and webinars. Across many initiatives the OCP Foundation and Community are meeting the market today and shaping the future. Learn more at: www.opencompute.org.

Appendix A. Composition Variant Summary

This Reference Architecture is accompanied by a companion repository [15] containing fully-specified composition variants with deployable artifacts. The repository is organized by size tier, with each variant providing a complete set of network design assets. The network designs are independent of cooling modality and rack density — the same composition applies to air-cooled and liquid-cooled deployments at any server-per-rack density. Operators should validate optic selections against site-specific cable distances.

Repository Structure

```
training-ra/compositions/  
├── OPG-64/  
│   ├── collapsed-conv--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/  
│   ├── clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/  
│   ├── mesh-conv-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g--inb-2x25g/  
│   └── mesh-conv-sh--cx7-1x400g--bf3-2x200g--storage-conv-2x200g--inb-2x25g/  
├── OPG-128/  
│   ├── clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/  
│   └── clos-sh--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/
```

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

```

└─ OPG-256/
|   └─ clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/
|   └─ dual-plane--cx8-2x400g--bf3-2x200g--2p--storage-conv-2x200g/
└─ OPG-512/
|   └─ clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/
|   └─ dual-plane--cx8-2x400g--bf3-2x200g--2p--storage-conv-2x200g/
└─ XOC-64/
|   └─ 1x-OPG-64/
|       └─ mesh-conv-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g--inb-2x25g/
|       └─ mesh-conv-sh--cx7-1x400g--bf3-2x200g--storage-conv-2x200g--inb-2x25g/
└─ XOC-256/
|   └─ 2x-OPG-128/
|       └─ clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/
|       └─ clos-sh--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/
└─ XOC-512/
|   └─ 1x-OPG-512/
|       └─ clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/
|       └─ dual-plane--cx8-2x400g--bf3-2x200g--2p--storage-conv-2x200g/
|   └─ 4x-OPG-128/
|       └─ clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/
|       └─ clos-sh--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/
└─ XOC-1024/
    └─ 2x-OPG-512/
        └─ clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g/
        └─ dual-plane--cx8-2x400g--bf3-2x200g--2p--storage-conv-2x200g/

```

Date: April 2026


 This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

The following table summarizes the 20 composition variants provided in the companion repository [15]:

OPG Building Blocks

Size	Variant	Topology	Scale-out NIC	Planes	Servers	xPUs
OPG-64	collapsed-conv	Collapsed converged (single leaf pair, no spine)	CX7 1×400G	1	8	64
OPG-64	clos-ro	Clos rail-optimized	CX7 1×400G	1	8	64
OPG-64	mesh-conv-ro	Mesh-converged rail-optimized	CX7 1×400G	1	8	64
OPG-64	mesh-conv-sh	Mesh-converged single-homed	CX7 1×400G	1	8	64
OPG-128	clos-ro	Clos rail-optimized	CX7 1×400G	1	16	128
OPG-128	clos-sh	Clos single-homed	CX7 1×400G	1	16	128
OPG-256	clos-ro	Clos rail-optimized	CX7 1×400G	1	32	256
OPG-256	dual-plane	Dual-plane rail-optimized	CX8 2×400G	2	32	256
OPG-512	clos-ro	Clos rail-optimized	CX7 1×400G	1	64	512
OPG-512	dual-plane	Dual-plane rail-optimized	CX8 2×400G	2	64	512

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

XOC Cluster Compositions

Tier	Composition	Variant	Topology	Scale-out NIC	Planes	Servers	xPUs
XOC-64	1× OPG-64	mesh-conv-ro	Mesh-converged rail-optimized	CX7 1×400G	1	8	64
XOC-64	1× OPG-64	mesh-conv-sh	Mesh-converged single-homed	CX7 1×400G	1	8	64
XOC-256	2× OPG-128	clos-ro	Clos rail-optimized	CX7 1×400G	1	32	256
XOC-256	2× OPG-128	clos-sh	Clos single-homed	CX7 1×400G	1	32	256
XOC-512	4× OPG-128	clos-ro	Clos rail-optimized	CX7 1×400G	1	64	512
XOC-512	4× OPG-128	clos-sh	Clos single-homed	CX7 1×400G	1	64	512
XOC-512	1× OPG-512	clos-ro	Clos rail-optimized	CX7 1×400G	1	64	512
XOC-512	1× OPG-512	dual-plane	Dual-plane rail-optimized	CX8 2×400G	2	64	512
XOC-1024	2× OPG-512	clos-ro	Clos rail-optimized	CX7 1×400G	1	128	1024
XOC-1024	2× OPG-512	dual-plane	Dual-plane rail-optimized	CX8 2×400G	2	128	1024

Date: April 2026


 This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

All variants share the following defaults unless otherwise noted: BlueField-3 2×200G frontend NICs with HostBGP multi-homing across two leaves, converged SO-C/storage, and Celestica DS5000 switches.

Topology Types

- **Clos rail-optimized (clos-ro):** Folded leaf-spine topology with server-to-leaf wiring constrained so that all NICs with the same GPU-affinity index (rail) connect to the same leaf or contiguous set of leaves. Same switch counts as single-homed Clos.
- **Clos single-homed (clos-sh):** Folded leaf-spine topology where all eight scale-out NICs from each xPU-node connect to the same leaf switch.
- **Dual-plane:** Two isolated, redundant scale-out Clos fabrics (Plane A and Plane B) using ConnectX-8 2×400G NICs (one port per plane). Doubles the first-hop rail domain size compared to single-plane.
- **Mesh-converged (mesh-conv):** A compact topology for small clusters (1–3 leaf switches) where leaf switches directly interconnect in a mesh without requiring separate spine switches. Available in rail-optimized and single-homed wiring patterns. Separate fabrics for scale-out, SO-C/storage, and in-band management.
- **Collapsed converged (collapsed-conv):** The simplest OPG topology — a single converged leaf pair carrying both scale-out (RDMA) backend and converged frontend (storage, in-band, management) traffic. No active spine; uplink ports are reserved for future XOC connectivity.

Assets Per Variant

Each variant folder contains a subset of the following assets, depending on maturity and complexity:

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

- **connectivity-map.csv** — End-to-end device and port mapping for every connection in the composition. Serves as the single source of truth for cabling, labeling, and validation.
- **topology-map.yaml** — Topology authoring plan used to generate inventories, wiring CRDs, and diagrams. Documents the design parameters and server/switch layout.
- **wiring/ directory** — Hedgehog Kubernetes CRD wiring definitions, one YAML file per Hedgehog-managed fabric. These files serve as the complete initial fabric configuration: operators supply these CRDs to the Hedgehog fabricator tool (hhfab) and the controller configures the entire fabric through zero-touch lifecycle management. Single-plane variants contain one backend wiring file and one frontend wiring file; dual-plane variants contain separate wiring files for each plane plus a frontend wiring file.
- **diagrams/ directory** — draw.io topology diagrams per fabric, providing visual reference for the composition layout.
- **netbox_inventory.json** — NetBox-importable inventory for integration with network management tooling.
- **bom.yaml** — Bill of materials with switch, optic, cable, and gateway quantities and specifications (available for select variants).
- **vpc.yaml** — Hedgehog VPC CRDs defining overlay networks, VPC attachments, and external connectivity patterns (available for select variants).

Not every variant includes all asset types. Each variant's README documents which assets are present and their status. Larger XOC tiers may include connectivity maps and topology plans while wiring CRDs and diagrams are in progress.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

Variant Naming Convention

Variants use a tokenized naming scheme for consistency and grep-ability:

```
<topology>--<scale-out-NIC>--<frontend-NIC>[--<planes>]--<storage>[--<in-band>]
```

Tokens:

- **Topology:** `clos-ro` (rail-optimized), `clos-sh` (single-homed), `dual-plane`, `mesh-conv-ro`, `mesh-conv-sh`, `collapsed-conv`
- **Scale-out NIC:** `cx7-1x400g` (ConnectX-7, 1×400G), `cx8-2x400g` (ConnectX-8, 2×400G)
- **Frontend NIC:** `bf3-2x200g` (BlueField-3, 2×200G)
- **Planes:** `2p` (dual-plane only)
- **Storage:** `storage-conv-2x200g` (converged SO-C/storage, 2×200G)
- **In-band (where explicit):** `inb-2x25g` (in-band management NIC speed)

Example: `clos-ro--cx7-1x400g--bf3-2x200g--storage-conv-2x200g`

XOC-Virtual (VLAB Composition)

The companion repository [15] includes an XOC-Virtual composition that provides a ready-to-use virtual lab topology. Operators can use this composition to:

- Instantiate a complete Hedgehog-managed fabric in the VLAB environment without physical switch hardware
- Practice the full lifecycle: ZTP enrollment, wiring CRD application, VPC overlay configuration, and Day-2 operations
- Explore the Hedgehog controller, `hhfctl` CLI, and Kubernetes CRD workflows hands-on

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

- Validate operational procedures and build team familiarity before production deployment

The XOC-Virtual uses the same CRD format and operational model as all other compositions in the repository. The virtual servers in VLAB provide a comparable network configuration to physical xPU-nodes, enabling a realistic (though not performance-representative) demonstration of the deployment and operational experience.

How to Use

1. Select a size tier matching your target xPU count (OPG-64 through XOC-1024).
2. Choose a variant based on your NIC configuration (single-plane vs dual-plane) and topology preference (rail-optimized vs single-homed). The network design is independent of cooling and density — the same composition applies at any server-per-rack density.
3. Review the connectivity map (connectivity-map.csv) to understand port assignments and cabling.
4. Apply Hedgehog Kubernetes CRDs from the wiring/ directory via hhfab or kubectl apply to configure the fabric.
5. Validate using the connectivity map against physical cabling and the BOM against procurement.
6. Adapt optics selections for site-specific distances post site-survey; defaults use flexible-reach DR/SR-class optics for broad compatibility.
7. Use the XOC-Virtual composition with Hedgehog VLAB for hands-on practice before physical deployment.

Date: April 2026



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).